Open in Colab

FIT5215: Deep Learning (2026)

CE/Lecturer (Clayton): **Dr Trung Le**
Lecturer (Clayton): **Dr Binh Nguyen**
Lecturer (Malaysia): **Dr Arghya Pal**, **Dr Sailaja Rajanala**, **Dr Sicily Fung Fung Ting**
Head Tutor (Clayton): **Mr Minh Vo** (FIT3181), **Dr Ruda Nie H** (FIT5215)

Faculty of Information Technology, Monash University, Australia

---

Student Information

Surname: **[Pham]**
Firstname: **[Nguyen]**
Student ID: **[30750814]**
Email: **[npha0026@student.monash.edu]**
Your tutorial time: **[Friday 2pm]**

Deep Neural Networks

Due: 11:55pm Sunday, 13 September 2026 (Sunday)

Important note: This is an **individual** assignment. It contributes **20%** to your final mark. Read the assignment instructions carefully.

What to submit

This assignment is to be completed individually and submitted to Moodle unit site. **By the due date, you are required to submit one single zip file, named xxx_assignment01_solution.zip where `xxx` is your student ID, to the corresponding Assignment (Dropbox) in Moodle**. You can use Google Colab to do Assigmnent 1 but you need to save it to an `*.ipynb` file to submit to the unit Moodle.

More importantly, if you use Google Colab to do this assignment, you need to first make a copy of this notebook on your Google drive.

For example, if your student ID is 123456, then gather all of your assignment solution to folder, create a zip file named 123456_assignment01_solution.zip and submit this file.

Within this zip folder, you **must** submit the following files:

1. **Assignment01_solution.ipynb**: this is your Python notebook solution source file.
2. **Assignment01_output.html** or **Assignment01_output.pdf**: this is the output of your Python notebook solution *exported* in html or pdf format.
3. Any **extra files or folder** needed to complete your assignment (e.g., images used in your answers).

Since the notebook is quite big to load and work together, one recommended option is to split solution into three parts and work on them seperately. In that case, replace **Assignment01_solution.ipynb** by three notebooks: **Assignment01_Part1_solution.ipynb**, **Assignment01_Part2_solution.ipynb** and **Assignment01_Part3_solution.ipynb**

You can run your codes on Google Colab. In this case, you have to make a copy of your Google colab notebook including the traces and progresses of model training before submitting.

Part 1: Theory and Knowledge Questions

[Total marks for this part: 30 points]

The first part of this assignment is to demonstrate your knowledge in deep learning that you have acquired from the lectures and tutorials materials. Most of the contents in this assignment are drawn from **the lectures and tutorials from weeks 1 to 4**. Going through these materials before attempting this part is highly recommended.

Question 1.1 Activation function plays an important role in modern Deep NNs. For each of the activation functions below, state its output range, find its derivative (show your steps), and plot the activation fuction and its derivative

(a) Exponential linear unit (ELU): $\mathrm{ELU}(x) = \begin{cases} 0.1\left(\exp(x) - 1\right) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$

[1.5 points]

(b) Gaussian Error Linear Unit (GELU): $\mathrm{GELU}(x) = x\Phi(x)$ where $\Phi(x)$ is the `probability cummulative function` of the standard Gaussian distribution or $\Phi(x) = \mathbb{P}\left(X \leq x\right)$ where $X \sim N\left(0, 1\right)$. In addition, the GELU activation fuction (the link for the main paper) has been widely used in the state-of-the-art Vision for Transformers (e.g., here is the link for the main ViT paper).

[1.5 points]

Question 1.1 Answer

AI USED TO REFORMAT ANSWER

(a) Exponential Linear Unit (ELU), $\alpha = 0.1$

$$\mathrm{ELU}(x) = \begin{cases} 0.1\,(\exp(x) - 1) & \text{if } x \le 0 \\ x & \text{if } x > 0 \end{cases}$$

Output range: $(-0.1, \infty)$. For $x > 0$ the output is x (unbounded above); for $x \le 0$ it is $0.1(\exp(x) - 1)$, which decreases towards -0.1 as $x \to -\infty$ but never reaches it.

Derivative (step by step):

- Left branch ($x < 0$): $\dfrac{d}{dx}\big(0.1\exp(x) - 0.1\big) = 0.1\exp(x)$
- Right branch ($x > 0$): $\dfrac{d}{dx}(x) = 1$

$$\mathrm{ELU}'(x) = \begin{cases} 0.1\exp(x) & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$

From the left branch, as x moves from $-\infty$ to 0 the derivative moves from 0 to 0.1, since $0.1\exp(-\infty) = 0.1 \times 0 = 0$ and $0.1\exp(0) = 0.1$. From the right branch the derivative is 1 for all $x > 0$.

Thus the derivative is **not defined at** $x = 0$, because the left-hand limit (0.1) does not match the right-hand limit (1). Therefore ELU with $\alpha = 0.1$ is differentiable everywhere **except** at $x = 0$. In practice ELU is commonly used with $\alpha = 1$, which makes the two one-sided derivatives agree at 0 and the function differentiable everywhere.

(b) Gaussian Error Linear Unit (GELU)

$$\mathrm{GELU}(x) = x\,\Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard Gaussian $X \sim N(0,1)$, and

$$g(x) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{x^2}{2}\right)$$

is the corresponding probability density function (PDF).

Output range: approximately $[-0.17, \infty)$. For $x > 0$, $\mathrm{GELU}(x) = x\Phi(x) > 0$ and grows like x (since $\Phi(x) \to 1$), so it is unbounded above. For $x < 0$ the output is negative, but $x\Phi(x) \to 0$ as $x \to -\infty$ because $\Phi(x)$ decays faster than $|x|$ grows, so unlike ReLU/ELU the negative part is a small bump rather than a line. The minimum is where $\mathrm{GELU}'(x) = 0$, i.e. $\Phi(x) + x\,g(x) = 0$, which is solved numerically at $x \approx -0.752$ giving $\mathrm{GELU}(x) \approx -0.170$. Hence the output is bounded below by about -0.17 (this is visible in the plot below).

Derivative (step by step):

Apply the product rule to $\mathrm{GELU}(x) = x\,\Phi(x)$:

$$\frac{d\,\mathrm{GELU}(x)}{dx} = x\,\Phi'(x) + \Phi(x)$$

By definition, the derivative of a CDF is its PDF:

$$\Phi'(x) = \frac{d\Phi(x)}{dx} = g(x)$$

Substituting back:

$$\frac{d\,\mathrm{GELU}(x)}{dx} = x\,g(x) + \Phi(x) = \frac{x}{\sqrt{2\pi}}\exp\left(-\frac{x^2}{2}\right) + \Phi(x)$$

Practical (tanh) approximation:

In practice $\mathrm{GELU}(x)$ is expensive to compute, because $\Phi(x)$ is an integral with no elementary closed form. In deep learning libraries GELU is therefore often approximated with a $\tanh$ expression:

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}}\left(x + 0.044715\,x^3\right)\right)\right)$$

which corresponds to approximating the CDF as

$$\Phi(x) \approx 0.5\left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}}\left(x + 0.044715\,x^3\right)\right)\right)$$

Substituting this approximation into the exact derivative gives the form used inside deep learning modules:

$$\frac{d\,\mathrm{GELU}(x)}{dx} \approx \frac{x}{\sqrt{2\pi}}\exp\left(-\frac{x^2}{2}\right) + 0.5\left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}}\left(x + 0.044715\,x^3\right)\right)\right)$$

In [1]:
```python
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" #to run on window machine

import torch
import torch.nn as nn
import matplotlib.pyplot as plt



# requires_grad=True so autograd can give us the derivatives

x = torch.linspace(-5.0, 5.0, 1001, requires_grad=True)

elu  = nn.ELU(alpha=0.1)
gelu = nn.GELU()          # exact (erf) form; nn.GELU(approximate='tanh') for
the tanh version

def value_and_grad(module, x):
  """f(x) and f'(x) elementwise. Summing is the trick: d(sum y)/dx_i =
dy_i/dx_i."""
  y = module(x)
  grad, = torch.autograd.grad(y.sum(), x, retain_graph=True)
  return y.detach(), grad

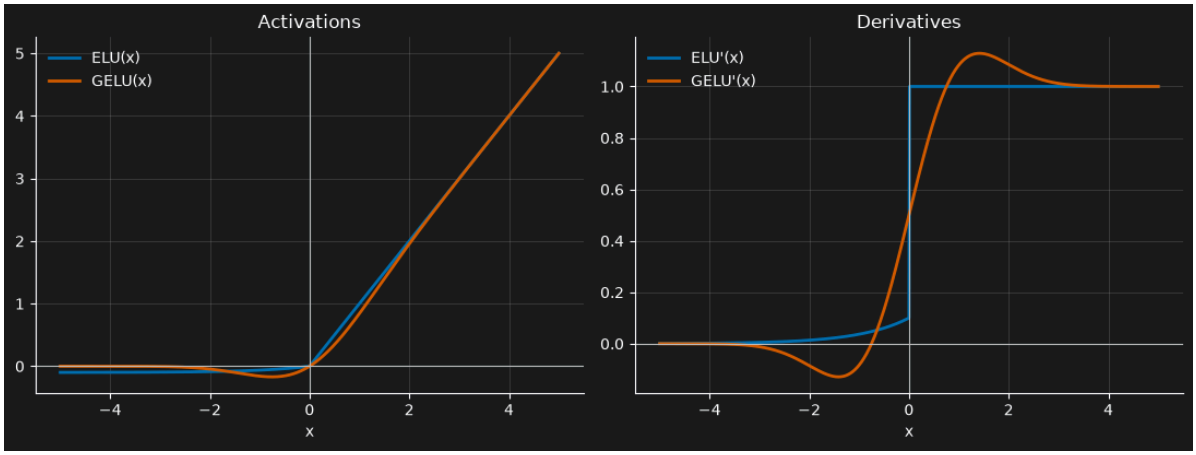
y_elu,  d_elu  = value_and_grad(elu,  x)
y_gelu, d_gelu = value_and_grad(gelu, x)
xs = x.detach()
```

```
BLUE, ORANGE = "#0072B2", "#D55E00"   # colourblind-safe pair
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharex=True)

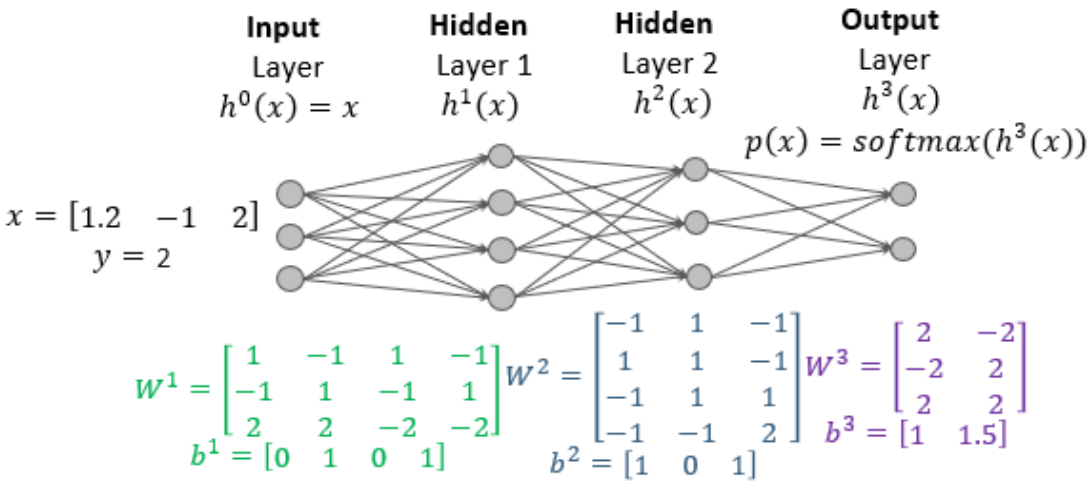
panels = [(y_elu, y_gelu, "ELU(x)",  "GELU(x)",  "Activations"),
          (d_elu, d_gelu, "ELU'(x)", "GELU'(x)", "Derivatives")]

for ax, (a, b, la, lb, title) in zip(axes, panels):
  ax.plot(xs, a, lw=2, color=BLUE,   label=la)
  ax.plot(xs, b, lw=2, color=ORANGE, label=lb)
  ax.axhline(0, lw=0.8, color="0.75")
  ax.axvline(0, lw=0.8, color="0.75")
  ax.grid(alpha=0.25, lw=0.6); ax.set_axisbelow(True)
  ax.set_title(title); ax.set_xlabel("x"); ax.legend(frameon=False)
  for s in ("top", "right"):
      ax.spines[s].set_visible(False)

fig.tight_layout()
plt.show()
```



Question 1.2: **Assume that we feed a data point x with a ground-truth label $y = 2$ to the feed-forward neural network with the `ReLU activation` function as shown in the following figure**



(a) What is the numerical value of the latent presentation $h^1(x)$?

[1 point]

(b) What is the numerical value of the latent presentation $h^2(x)$?

[1 point]

(c) What is the numerical value of the logit $h^3(x)$?

[1 point]

(d) What is the corresonding prediction probabilities $p(x)$?

[1 point]

(e) What is the predicted label $\hat{y}$? Is it a correct and an incorect prediction? Remind that $y = 2$.

[1 point]

(f) What is the cross-entropy loss caused by the feed-forward neural network at (x, y)? Remind that $y = 2$.

[1 point]

(g) Why is the cross-entropy loss caused by the feed-forward neural network at (x, y) (i.e., $\mathrm{CE}(1_y, p(x))$) always non-negative? When does this $\mathrm{CE}(1_y, p(x))$ loss get the value 0? Note that you need to answer this question for a general pair (x, y) and a general feed-forward neural network with, for example $M = 4$ classes?

[1 point]

You must show both formulas and numerical results for earning full mark. Although it is optional, it is great if you show your PyTorch code for your computation.

Q1.2 Answer

a) h1(x)= RELU (W1* x +b1) = [6.2000, 2.8000, 0.0000, 0.0000]

b) h2(x) = RELU (W2* h1 +b2) = [0., 9., 0.]

c) h3(logits) = W3*h2 + b3 = [-17.0000, 19.5000]

d) p(x) = softmax(h3) = [1.4069e-16, 1.0000e+00], rounding error occurs here the logit almost make a perfect prediction [0,1]

e) The predicted label $\hat{y}$? = 2, this is a correct prediction

f) The cross entropy loss = -log(1- 1.4069e-16) ~=0, this is from: y[0,1] p[[1.4069e-16, 1.0000e+00] CE(x,y)= y*-log(p(x))

g) CE is always non-negative because: CE calculate at truth label and its correlated predicted label because at truth label y=1 and y=0 everywhere else so only at truth label matter, CE= 1*-log(p(x)), where p(x) is the probability between [0,1]. here we can see that the value of CE falls between -log(0) and -log(1) so [0, inf). When CE(x,y)= 0, we have p(x) =1, which means the model predict this label with 100% certainty and is correct. For CE network of 4 classes the CE is calculated as follow:

CE(x,y)= Sum(i=1 to 4) yi * -log(xi), where yi is the ground truth value [0 or 1] and xi is the predicted probability value range of [0,1]

In [2]:
```python
import torch

# Question 1.2 - parameters read from the network figure
# Architecture: 3 (input) -> 4 (ReLU) -> 3 (ReLU) -> 2 (output/logits)
# x is a row vector, so each layer is computed as  h = x @ W + b

x = torch.tensor([1.2, -1.0, 2.0])          # shape (3,)
y = 2                                        # ground-truth label (class 2)

W1 = torch.tensor([[ 1.0, -1.0,  1.0, -1.0],
                   [-1.0,  1.0, -1.0,  1.0],
                   [ 2.0,  2.0, -2.0, -2.0]])   # shape (3, 4)
b1 = torch.tensor([0.0, 1.0, 0.0, 1.0])         # shape (4,)

W2 = torch.tensor([[-1.0,  1.0, -1.0],
                   [ 1.0,  1.0, -1.0],
                   [-1.0,  1.0,  1.0],
                   [-1.0, -1.0,  2.0]])         # shape (4, 3)
b2 = torch.tensor([1.0, 0.0, 1.0])              # shape (3,)

W3 = torch.tensor([[ 2.0, -2.0],
                   [-2.0,  2.0],
                   [ 2.0,  2.0]])               # shape (3, 2)
b3 = torch.tensor([1.0, 1.5])                   # shape (2,)

for name, t in [("x", x), ("W1", W1), ("b1", b1), ("W2", W2),
                ("b2", b2), ("W3", W3), ("b3", b3)]:
    print(f"{name}: shape {tuple(t.shape)}")

h1 =  torch.relu(x.matmul(W1)+b1)
print("h1= ",h1)

h2 = torch.relu(h1.matmul(W2)+b2)
print("h2= ",h2)

h3 = h2.matmul(W3) +b3
print("h3 logits= ", h3)

px = torch.softmax(h3, dim=0)
print("px = ",px)
```

```
x: shape (3,)
W1: shape (3, 4)
b1: shape (4,)
W2: shape (4, 3)
b2: shape (3,)
W3: shape (3, 2)
b3: shape (2,)
h1=  tensor([6.2000, 2.8000, 0.0000, 0.0000])
h2=  tensor([0., 9., 0.])
h3 logits=  tensor([-17.0000,  19.5000])
px =  tensor([1.4069e-16, 1.0000e+00])
```

In [ ]:
```
```

Question 1.3:

For **Question 1.3**, you have two options:

- **(1)** *perform the forward, backward propagation, and SGD update for* `one mini-batch` (**10 points**), or
- **(2)** *manually implement a feed-forward neural network* that can work on real tabular datasets (**20 points**).

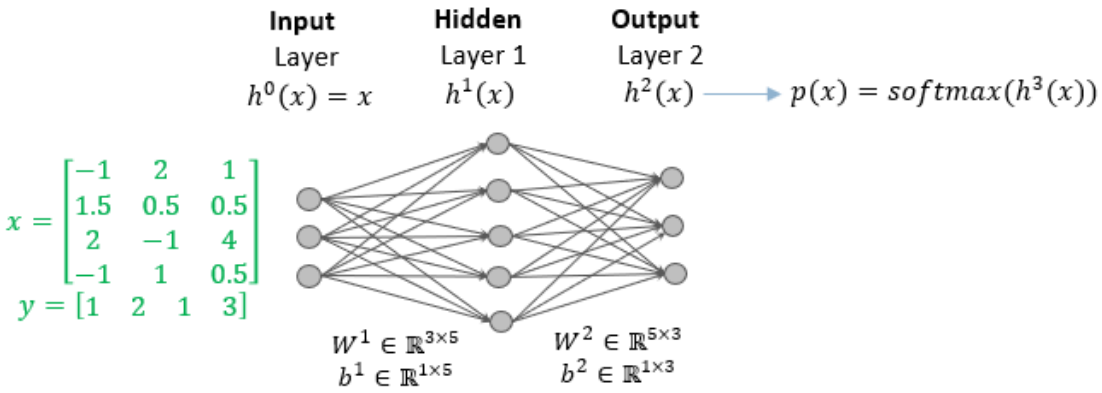
You can choose either **(1)** or **(2)** to proceed.

Option 1

[Total marks for this option: 10 points]

Assume that we are constructing a multilayered feed-forward neural network for a classification problem with three classes where the model parameters will be generated randomly using your student ID. The architecture of this network is $3(Input) \rightarrow 5(ELU) \rightarrow 3(Output)$ as shown in the following figure. Note that the ELU has the same formula as the one in Q1.1.

We feed a batch X with the labels Y as shown in the figure. Answer the following questions.



You need to show both formulas, numerical results, and your PyTorch code for your computation for earning full marks.

In [2]:

In [3]:
```
#Code to generate random matrices and biases for W1, b1, W2, b2
```

Forward propagation

(a) What is the value of $\bar{h}^1(x)$ (the pre-activation values of h^1)?

[0.5 point]

In [4]:
```
#Show your code
```

(b) What is the value of $h^1(x)$?

[0.5 point]

In [5]:
```
#Show your code
```

(c) What is the predicted value $\hat{y}$?

[0.5 point]

In [6]:
```
#Show your code
```

(d) Suppose that we use the cross-entropy (CE) loss. What is the value of the CE loss l incurred by the mini-batch?

[0.5 point]

In [7]:
```
#Show your code
```

Backward propagation

(e) What are the derivatives $\frac{\partial l}{\partial h^2}$, $\frac{\partial l}{\partial W^2}$, and $\frac{\partial l}{\partial b^2}$?

[3 points]

In [8]:
```
#Show your code
```

(f) What are the derivatives $\frac{\partial l}{\partial h^1}$, $\frac{\partial l}{\partial \bar{h}^1}$, $\frac{\partial l}{\partial W^1}$, and $\frac{\partial l}{\partial b^1}$?

[3 points]

In [9]:
```
#Show your code
```

SGD update

(g) Assume that we use SGD with learning rate $\eta = 0.01$ to update the model parameters. What are the values of W^2, b^2 and W^1, b^1 after updating?

[2 points]

In [10]:
```
#Show your code
```

Option 2

[Total marks for this option: 20 points]

In [20]:
```
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

In Option 2, you need to implement a feed-forward NN manually using PyTorch and auto-differentiation of PyTorch. We then manually train the model on the MNIST dataset.

We first download the `MNIST` dataset and preprocess it.

In [21]:
```
transform = transforms.Compose([
    transforms.ToTensor(),  # Convert the image to a tensor with shape [C, H, W]
    transforms.Normalize((0.5,), (0.5,)),  # Normalize to [-1, 1]
    transforms.Lambda(lambda x: x.view(28*28)) # Flatten the tensor to shape
[-1,HW]
])

# Load the MNIST dataset
train_dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

train_data, train_labels = train_dataset.data, train_dataset.targets
test_data, test_labels = test_dataset.data, test_dataset.targets
print(train_data.shape, train_labels.shape)
print(test_data.shape, test_labels.shape)
```

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
```

Each data point has dimension `[28,28]`. We need to flatten it to a vector to input to our FFN.

In [22]:
```
train_dataset.data = train_data.data.reshape(-1, 28*28)
test_dataset.data = test_data.data.reshape(-1, 28*28)

train_data, train_labels = train_dataset.data, train_dataset.targets
test_data, test_labels = test_dataset.data, test_dataset.targets
print(train_data.shape, train_labels.shape)
print(test_data.shape, test_labels.shape)
```

```
torch.Size([60000, 784]) torch.Size([60000])
torch.Size([10000, 784]) torch.Size([10000])
```

We split the train and test sets into many mini-batches of 64.

In [23]:
```
train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=64, shuffle=False)
```

Develop the feed-forward neural networks

(a) You need to develop the class `MyLinear` with the following skeleton. You need to declare the weight matrix and bias of this linear layer.

[3 points]

In [24]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class MyLinear(torch.nn.Module):
  def __init__(self, input_size, output_size):
    """
    input_size: the size of the input
    output_size: the size of the output
    """
    super().__init__()
    #Your code here
    # A linear layer computes  h = x @ W + b  with
```

```
    #   W : weight matrix of shape [input_size, output_size]
    #   b : bias vector  of shape [output_size]
    # Both are wrapped in torch.nn.Parameter so that PyTorch registers them as
trainable
    # parameters of the module (model.parameters(), model.zero_grad(),
state_dict() all work).
    # W is initialised with N(0, 1/input_size) (LeCun initialisation) so that
the
    # pre-activations keep a unit scale, b is initialised with zeros.
    self.W = torch.nn.Parameter(torch.randn(input_size, output_size,
device=device) / (input_size ** 0.5))
    self.b = torch.nn.Parameter(torch.zeros(output_size, device=device))

  #forward propagation
  def forward(self, x): #x is a mini-batch
    #Your code here
    # x has shape [batch_size, input_size]  ->  output has shape [batch_size,
output_size]
    return torch.matmul(x, self.W) + self.b
```

(b) You need to develop the class `MyFFN` with the following skeleton

[7 points]

In [16]:
```
class MyFFN(torch.nn.Module):
  def __init__(self, input_size, num_classes, hidden_sizes, act =
torch.nn.ReLU()):
    """
    input_size: the size of the input
    num_classes: the number of classes
    act is the activation function
    hidden_sizes is the list of hidden sizes
    for example input_size = 3, hidden_sizes = [5, 7], num_classes = 4, and act
= torch.nn.ReLU()
    means that we are building up a FFN with the confirguration
    (3 (Input) -> 5 (ReLU) -> 7 (ReLU) -> 4 (Output))
    """
    super(MyFFN, self).__init__()
    self.input_size = input_size
    self.num_classes = num_classes
    self.act = act
    self.hidden_sizes = hidden_sizes
    self.num_layers = len(hidden_sizes) + 1

  def _make_act(self):
    """
    `act` can be passed either as a class (torch.nn.ReLU) or as an instance
(torch.nn.ReLU()).
    Return a module in both cases.
    """
    return self.act() if isinstance(self.act, type) else self.act

  def create_FFN(self):
    """
    This function creates the feed-forward neural network
    We stack many MyLinear layers
    """
    hidden_sizes = [self.input_size] + self.hidden_sizes + [self.num_classes]
    self.layers = torch.nn.ModuleList()
```

```
    #Your code here
    # Hidden layers: each one is a MyLinear followed by the activation function.
    # With hidden_sizes = [784, 100, 100, 10] this builds 784 -> 100 (act) ->
100 (act)
    for index in range(len(self.hidden_sizes)):
        layer = torch.nn.Sequential(MyLinear(hidden_sizes[index],
hidden_sizes[index + 1]),
                                    self._make_act())
        self.layers.append(layer)
    # Output layer: a MyLinear WITHOUT activation (100 -> 10). It returns the
logits;
    # the softmax is applied inside the cross-entropy loss.
    self.layers.append(MyLinear(hidden_sizes[-2], hidden_sizes[-1]))

  def forward(self, x):
    """
    This implements the forward propagation of the batch x
    This needs to return the logits of x
    """
    result = x
    for layer in self.layers:
        result = layer(result)
    return result

  def compute_loss(self, x, y):
    """
    This function computes the cross-entropy loss
    You can use the built-in CE loss of PyTorch
    """
    #Your code here
    logits = self.forward(x)                # shape [batch_size, num_classes]
    loss_fn = torch.nn.CrossEntropyLoss()  # = softmax + negative log-
likelihood, averaged over the batch
    return loss_fn(logits, y)

  # ---- helpers shared by the three optimisers ------------------------------
-----------
  def _linear_layers(self):
    """All MyLinear layers of the network (the ones that own the parameters W
and b)."""
    return [m for m in self.modules() if isinstance(m, MyLinear)]

  def _backward(self, x, y):
    """
    Common first step of every update rule:
    clear the gradients of the previous step, compute the loss on the batch (x,
y)
    and back-propagate to obtain dl/dW and dl/db for every layer.
    """
    self.train()
    self.zero_grad()
    loss = self.compute_loss(x, y)
    loss.backward()
    return loss


  def update_SGD(self, x, y, learning_rate = 0.01):
    """
    This function updates the model parameters using SGD using the batch (x,y)
    You need to implement the update rule manually and cannot rely on the built-
```

```
in optimizer

    Update rule, for every parameter theta with gradient g = dl/dtheta on the
mini-batch:
        theta <- theta - eta * g
    """
    #Your code here
    self._backward(x, y)
    with torch.no_grad():
        for layer in self._linear_layers():
            layer.W -= learning_rate * layer.W.grad
            layer.b -= learning_rate * layer.b.grad


  def update_SGDwithMomentum(self, x, y, learning_rate = 0.01, momentum = 0.9):
    """
    This function updates the model parameters using SGD with momentum using the
batch (x,y)
    You need to implement the update rule manually and cannot rely on the built-
in optimizer

    Update rule (same form as Tutorial 4, Exercise 2). The velocity v is an
exponential
    moving average of the past gradients, initialised with zeros:
        v     <- alpha * v + (1 - alpha) * g
        theta <- theta - eta * v
    (torch.optim.SGD uses v <- alpha * v + g, i.e. the same rule up to a
rescaling of eta.)
    The velocities are stored on each MyLinear layer as vW and vb.
    """
    #Your code here
    self._backward(x, y)
    with torch.no_grad():
        for layer in self._linear_layers():
            if not hasattr(layer, 'vW'):                # first call: v = 0
                layer.vW = torch.zeros_like(layer.W)
                layer.vb = torch.zeros_like(layer.b)
            layer.vW.mul_(momentum).add_(layer.W.grad, alpha=1.0 - momentum)
            layer.vb.mul_(momentum).add_(layer.b.grad, alpha=1.0 - momentum)
            layer.W -= learning_rate * layer.vW
            layer.b -= learning_rate * layer.vb


  def update_AdaGrad(self, x, y, learning_rate = 0.01, epsilon = 1e-8):
    """
    This function updates the model parameters using AdaGrad using the batch
(x,y)
    You need to implement the update rule manually and cannot rely on the built-
in optimizer

    Update rule. AdaGrad keeps, for every parameter, the sum s of all squared
gradients seen
    so far and uses it to scale the learning rate element-wise (parameters that
received large
    gradients get smaller steps):
        s     <- s + g * g
        theta <- theta - eta * g / (sqrt(s) + epsilon)
    The accumulators are stored on each MyLinear layer as sW and sb, initialised
with zeros.
    """
```

```
    #Your code here
    self._backward(x, y)
    with torch.no_grad():
        for layer in self._linear_layers():
            if not hasattr(layer, 'sW'):                # first call: s = 0
                layer.sW = torch.zeros_like(layer.W)
                layer.sb = torch.zeros_like(layer.b)
            layer.sW.addcmul_(layer.W.grad, layer.W.grad)   # s += g * g
            layer.sb.addcmul_(layer.b.grad, layer.b.grad)
            layer.W -= learning_rate * layer.W.grad / (torch.sqrt(layer.sW) +
epsilon)
            layer.b -= learning_rate * layer.b.grad / (torch.sqrt(layer.sb) +
epsilon)
```

In [17]:
```
myFFN = MyFFN(input_size = 28*28, num_classes = 10, hidden_sizes = [100, 100],
act = torch.nn.ReLU)
myFFN.create_FFN()
print(myFFN)
```

```
MyFFN(
  (layers): ModuleList(
    (0-1): 2 x Sequential(
      (0): MyLinear()
      (1): ReLU()
    )
    (2): MyLinear()
  )
)
```

(c) Write the code to evaluate the accuracy of the current `myFFN` model on a data loader (e.g., train_loader or test_loader).

[2.5 points]

In [18]:
```
def compute_acc(model, data_loader):
  """
  This function computes the accuracy of the model on a data loader
  """
  #Your code here
  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    # Change the model to the evaluation mode (important for the model with
batch-norm and drop-out layers)
  model.eval()
  # In test phase, we don't need to compute gradients (for memory efficiency)
  with torch.no_grad():
    corrects = 0
    totals =0
    for (batchX, batchY) in data_loader:
      batchX, batchY = batchX.to(device), batchY.to(device)
      outputs = model(batchX.type(torch.float32)) #feed batch to the model
      totals += batchY.size(0) #accumulate totals with the current batch size
      predicted = torch.argmax(outputs.data, 1) #get the predicted class
      corrects += (predicted == batchY.type(torch.long)).sum().item()
#accumulate correct predictions
  acc = float(corrects)/totals #compute the accuracy
  return acc
```

(d) Write the code to evaluate the loss of the current `myFFN` model on a data loader (e.g., train_loader or test_loader).

[2.5 points]

In [19]:
```
def compute_loss(model, data_loader):

  model.eval()
  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
  loss_fn = torch.nn.CrossEntropyLoss()
  loss = 0
  with torch.no_grad():
    for (batchX, batchY) in data_loader:
      batchX, batchY = batchX.to(device), batchY.to(device)
      outputs = model(batchX.type(torch.float32))
      loss += loss_fn(outputs, batchY.type(torch.long)).item()
  return loss/len(data_loader)
```

Train on the `MNIST` data with 50 epochs using `updateSGD`.

In [20]:
```
num_epochs = 50
for epoch in range(num_epochs):
    for i, (x, y) in enumerate(train_loader):
      x, y = x.to(device), y.to(device)
      myFFN.update_SGD(x, y, learning_rate = 0.01)
    train_acc = compute_acc(myFFN, train_loader)
    train_loss = compute_loss(myFFN, train_loader)
    test_acc = compute_acc(myFFN, test_loader)
    test_loss = compute_loss(myFFN, test_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f}, Train
Acc: {train_acc*100:.2f}%, Test Loss: {test_loss:.4f}, Test Acc:
{test_acc*100:.2f}%")
```

```
Epoch 1/50, Train Loss: 0.3691, Train Acc: 89.26%, Test Loss: 0.3578, Test Acc:
89.40%
Epoch 2/50, Train Loss: 0.2959, Train Acc: 91.23%, Test Loss: 0.2819, Test Acc:
91.72%
Epoch 3/50, Train Loss: 0.2697, Train Acc: 92.19%, Test Loss: 0.2627, Test Acc:
92.18%
Epoch 4/50, Train Loss: 0.2325, Train Acc: 93.29%, Test Loss: 0.2301, Test Acc:
93.21%
Epoch 5/50, Train Loss: 0.2196, Train Acc: 93.47%, Test Loss: 0.2169, Test Acc:
93.33%
Epoch 6/50, Train Loss: 0.1948, Train Acc: 94.16%, Test Loss: 0.1964, Test Acc:
93.63%
Epoch 7/50, Train Loss: 0.1725, Train Acc: 95.00%, Test Loss: 0.1758, Test Acc:
94.59%
Epoch 8/50, Train Loss: 0.1561, Train Acc: 95.46%, Test Loss: 0.1609, Test Acc:
94.97%
Epoch 9/50, Train Loss: 0.1462, Train Acc: 95.76%, Test Loss: 0.1525, Test Acc:
95.28%
Epoch 10/50, Train Loss: 0.1461, Train Acc: 95.78%, Test Loss: 0.1520, Test Acc:
95.31%
Epoch 11/50, Train Loss: 0.1269, Train Acc: 96.32%, Test Loss: 0.1359, Test Acc:
95.65%
Epoch 12/50, Train Loss: 0.1203, Train Acc: 96.62%, Test Loss: 0.1318, Test Acc:
95.82%
Epoch 13/50, Train Loss: 0.1137, Train Acc: 96.67%, Test Loss: 0.1260, Test Acc:
96.08%
Epoch 14/50, Train Loss: 0.1079, Train Acc: 96.91%, Test Loss: 0.1224, Test Acc:
96.19%
Epoch 15/50, Train Loss: 0.1077, Train Acc: 96.84%, Test Loss: 0.1202, Test Acc:
96.21%
Epoch 16/50, Train Loss: 0.0997, Train Acc: 97.14%, Test Loss: 0.1162, Test Acc:
96.45%
Epoch 17/50, Train Loss: 0.0933, Train Acc: 97.30%, Test Loss: 0.1105, Test Acc:
96.56%
Epoch 18/50, Train Loss: 0.0889, Train Acc: 97.47%, Test Loss: 0.1085, Test Acc:
96.68%
Epoch 19/50, Train Loss: 0.0844, Train Acc: 97.61%, Test Loss: 0.1082, Test Acc:
96.60%
Epoch 20/50, Train Loss: 0.0816, Train Acc: 97.62%, Test Loss: 0.1048, Test Acc:
96.68%
Epoch 21/50, Train Loss: 0.0788, Train Acc: 97.69%, Test Loss: 0.1022, Test Acc:
96.75%
Epoch 22/50, Train Loss: 0.0779, Train Acc: 97.73%, Test Loss: 0.1027, Test Acc:
96.81%
Epoch 23/50, Train Loss: 0.0709, Train Acc: 97.99%, Test Loss: 0.0951, Test Acc:
97.10%
Epoch 24/50, Train Loss: 0.0655, Train Acc: 98.09%, Test Loss: 0.0928, Test Acc:
97.10%
Epoch 25/50, Train Loss: 0.0657, Train Acc: 98.12%, Test Loss: 0.0930, Test Acc:
97.00%
Epoch 26/50, Train Loss: 0.0691, Train Acc: 97.87%, Test Loss: 0.0973, Test Acc:
96.87%
Epoch 27/50, Train Loss: 0.0610, Train Acc: 98.27%, Test Loss: 0.0908, Test Acc:
97.23%
Epoch 28/50, Train Loss: 0.0628, Train Acc: 98.18%, Test Loss: 0.0947, Test Acc:
96.98%
Epoch 29/50, Train Loss: 0.0581, Train Acc: 98.28%, Test Loss: 0.0883, Test Acc:
97.25%
Epoch 30/50, Train Loss: 0.0529, Train Acc: 98.54%, Test Loss: 0.0866, Test Acc:
97.26%
```

```
Epoch 31/50, Train Loss: 0.0493, Train Acc: 98.64%, Test Loss: 0.0846, Test Acc:
97.28%
Epoch 32/50, Train Loss: 0.0486, Train Acc: 98.62%, Test Loss: 0.0823, Test Acc:
97.30%
Epoch 33/50, Train Loss: 0.0457, Train Acc: 98.70%, Test Loss: 0.0811, Test Acc:
97.43%
Epoch 34/50, Train Loss: 0.0477, Train Acc: 98.66%, Test Loss: 0.0845, Test Acc:
97.24%
Epoch 35/50, Train Loss: 0.0439, Train Acc: 98.76%, Test Loss: 0.0835, Test Acc:
97.40%
Epoch 36/50, Train Loss: 0.0447, Train Acc: 98.77%, Test Loss: 0.0846, Test Acc:
97.29%
Epoch 37/50, Train Loss: 0.0479, Train Acc: 98.60%, Test Loss: 0.0900, Test Acc:
97.24%
Epoch 38/50, Train Loss: 0.0419, Train Acc: 98.82%, Test Loss: 0.0825, Test Acc:
97.36%
Epoch 39/50, Train Loss: 0.0399, Train Acc: 98.93%, Test Loss: 0.0825, Test Acc:
97.47%
Epoch 40/50, Train Loss: 0.0378, Train Acc: 98.96%, Test Loss: 0.0795, Test Acc:
97.58%
Epoch 41/50, Train Loss: 0.0380, Train Acc: 98.91%, Test Loss: 0.0801, Test Acc:
97.42%
Epoch 42/50, Train Loss: 0.0365, Train Acc: 98.99%, Test Loss: 0.0780, Test Acc:
97.51%
Epoch 43/50, Train Loss: 0.0363, Train Acc: 99.01%, Test Loss: 0.0832, Test Acc:
97.42%
Epoch 44/50, Train Loss: 0.0321, Train Acc: 99.18%, Test Loss: 0.0786, Test Acc:
97.49%
Epoch 45/50, Train Loss: 0.0442, Train Acc: 98.60%, Test Loss: 0.0927, Test Acc:
96.97%
Epoch 46/50, Train Loss: 0.0321, Train Acc: 99.15%, Test Loss: 0.0797, Test Acc:
97.53%
Epoch 47/50, Train Loss: 0.0332, Train Acc: 99.10%, Test Loss: 0.0857, Test Acc:
97.31%
Epoch 48/50, Train Loss: 0.0314, Train Acc: 99.13%, Test Loss: 0.0811, Test Acc:
97.47%
Epoch 49/50, Train Loss: 0.0274, Train Acc: 99.31%, Test Loss: 0.0772, Test Acc:
97.70%
Epoch 50/50, Train Loss: 0.0314, Train Acc: 99.11%, Test Loss: 0.0829, Test Acc:
97.40%
```

(e) Implement the function `updateSGDMomentum` in the class and train the model with this optimizer in `50` epochs. You can update the corresponding function in the `MyFNN` class.

[2.5 points]

In [21]:

```python
#Your code here
myFFN = MyFFN(input_size = 28*28, num_classes = 10, hidden_sizes = [100, 100],
act = torch.nn.ReLU)
myFFN.create_FFN()
num_epochs = 50
for epoch in range(num_epochs):
    for i, (x, y) in enumerate(train_loader):
      x, y = x.to(device), y.to(device)
      myFFN.update_SGDwithMomentum(x, y, learning_rate = 0.01)
    train_acc = compute_acc(myFFN, train_loader)
    train_loss = compute_loss(myFFN, train_loader)
    test_acc = compute_acc(myFFN, test_loader)
```

```
    test_loss = compute_loss(myFFN, test_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f}, Train
Acc: {train_acc*100:.2f}%, Test Loss: {test_loss:.4f}, Test Acc:
{test_acc*100:.2f}%")
```

```
Epoch 1/50, Train Loss: 0.3572, Train Acc: 89.82%, Test Loss: 0.3415, Test Acc: 90.57%
Epoch 2/50, Train Loss: 0.2970, Train Acc: 91.25%, Test Loss: 0.2866, Test Acc: 91.63%
Epoch 3/50, Train Loss: 0.2621, Train Acc: 92.28%, Test Loss: 0.2570, Test Acc: 92.46%
Epoch 4/50, Train Loss: 0.2386, Train Acc: 93.13%, Test Loss: 0.2365, Test Acc: 93.28%
Epoch 5/50, Train Loss: 0.2133, Train Acc: 93.89%, Test Loss: 0.2114, Test Acc: 93.79%
Epoch 6/50, Train Loss: 0.1948, Train Acc: 94.39%, Test Loss: 0.1970, Test Acc: 94.07%
Epoch 7/50, Train Loss: 0.1802, Train Acc: 94.86%, Test Loss: 0.1836, Test Acc: 94.65%
Epoch 8/50, Train Loss: 0.1631, Train Acc: 95.30%, Test Loss: 0.1703, Test Acc: 95.04%
Epoch 9/50, Train Loss: 0.1543, Train Acc: 95.50%, Test Loss: 0.1619, Test Acc: 95.15%
Epoch 10/50, Train Loss: 0.1403, Train Acc: 96.00%, Test Loss: 0.1492, Test Acc: 95.64%
Epoch 11/50, Train Loss: 0.1307, Train Acc: 96.25%, Test Loss: 0.1406, Test Acc: 95.97%
Epoch 12/50, Train Loss: 0.1229, Train Acc: 96.41%, Test Loss: 0.1358, Test Acc: 96.10%
Epoch 13/50, Train Loss: 0.1161, Train Acc: 96.59%, Test Loss: 0.1306, Test Acc: 96.23%
Epoch 14/50, Train Loss: 0.1110, Train Acc: 96.76%, Test Loss: 0.1248, Test Acc: 96.33%
Epoch 15/50, Train Loss: 0.1037, Train Acc: 97.00%, Test Loss: 0.1212, Test Acc: 96.37%
Epoch 16/50, Train Loss: 0.0976, Train Acc: 97.12%, Test Loss: 0.1165, Test Acc: 96.43%
Epoch 17/50, Train Loss: 0.0962, Train Acc: 97.20%, Test Loss: 0.1167, Test Acc: 96.69%
Epoch 18/50, Train Loss: 0.0887, Train Acc: 97.46%, Test Loss: 0.1118, Test Acc: 96.71%
Epoch 19/50, Train Loss: 0.0845, Train Acc: 97.56%, Test Loss: 0.1044, Test Acc: 96.82%
Epoch 20/50, Train Loss: 0.0795, Train Acc: 97.71%, Test Loss: 0.1042, Test Acc: 96.87%
Epoch 21/50, Train Loss: 0.0768, Train Acc: 97.70%, Test Loss: 0.0999, Test Acc: 96.96%
Epoch 22/50, Train Loss: 0.0758, Train Acc: 97.80%, Test Loss: 0.1029, Test Acc: 96.85%
Epoch 23/50, Train Loss: 0.0713, Train Acc: 97.91%, Test Loss: 0.0968, Test Acc: 97.09%
Epoch 24/50, Train Loss: 0.0667, Train Acc: 98.09%, Test Loss: 0.0958, Test Acc: 97.05%
Epoch 25/50, Train Loss: 0.0652, Train Acc: 98.17%, Test Loss: 0.0949, Test Acc: 97.21%
Epoch 26/50, Train Loss: 0.0673, Train Acc: 98.05%, Test Loss: 0.0966, Test Acc: 97.04%
Epoch 27/50, Train Loss: 0.0617, Train Acc: 98.24%, Test Loss: 0.0944, Test Acc: 97.03%
Epoch 28/50, Train Loss: 0.0617, Train Acc: 98.21%, Test Loss: 0.0956, Test Acc: 97.08%
Epoch 29/50, Train Loss: 0.0554, Train Acc: 98.47%, Test Loss: 0.0893, Test Acc: 97.22%
Epoch 30/50, Train Loss: 0.0571, Train Acc: 98.40%, Test Loss: 0.0921, Test Acc: 97.25%
```

```
Epoch 31/50, Train Loss: 0.0513, Train Acc: 98.59%, Test Loss: 0.0860, Test Acc:
97.41%
Epoch 32/50, Train Loss: 0.0515, Train Acc: 98.55%, Test Loss: 0.0887, Test Acc:
97.29%
Epoch 33/50, Train Loss: 0.0500, Train Acc: 98.56%, Test Loss: 0.0858, Test Acc:
97.36%
Epoch 34/50, Train Loss: 0.0476, Train Acc: 98.72%, Test Loss: 0.0842, Test Acc:
97.44%
Epoch 35/50, Train Loss: 0.0452, Train Acc: 98.72%, Test Loss: 0.0833, Test Acc:
97.51%
Epoch 36/50, Train Loss: 0.0443, Train Acc: 98.81%, Test Loss: 0.0838, Test Acc:
97.51%
Epoch 37/50, Train Loss: 0.0428, Train Acc: 98.85%, Test Loss: 0.0854, Test Acc:
97.35%
Epoch 38/50, Train Loss: 0.0414, Train Acc: 98.89%, Test Loss: 0.0817, Test Acc:
97.50%
Epoch 39/50, Train Loss: 0.0407, Train Acc: 98.91%, Test Loss: 0.0830, Test Acc:
97.43%
Epoch 40/50, Train Loss: 0.0389, Train Acc: 98.98%, Test Loss: 0.0818, Test Acc:
97.41%
Epoch 41/50, Train Loss: 0.0356, Train Acc: 99.09%, Test Loss: 0.0774, Test Acc:
97.66%
Epoch 42/50, Train Loss: 0.0372, Train Acc: 99.02%, Test Loss: 0.0818, Test Acc:
97.54%
Epoch 43/50, Train Loss: 0.0339, Train Acc: 99.13%, Test Loss: 0.0798, Test Acc:
97.65%
Epoch 44/50, Train Loss: 0.0334, Train Acc: 99.17%, Test Loss: 0.0809, Test Acc:
97.56%
Epoch 45/50, Train Loss: 0.0351, Train Acc: 99.07%, Test Loss: 0.0804, Test Acc:
97.69%
Epoch 46/50, Train Loss: 0.0308, Train Acc: 99.28%, Test Loss: 0.0776, Test Acc:
97.73%
Epoch 47/50, Train Loss: 0.0320, Train Acc: 99.18%, Test Loss: 0.0810, Test Acc:
97.51%
Epoch 48/50, Train Loss: 0.0319, Train Acc: 99.15%, Test Loss: 0.0806, Test Acc:
97.65%
Epoch 49/50, Train Loss: 0.0294, Train Acc: 99.28%, Test Loss: 0.0785, Test Acc:
97.64%
Epoch 50/50, Train Loss: 0.0277, Train Acc: 99.36%, Test Loss: 0.0768, Test Acc:
97.71%
```

(f) Implement the function `updateAdagrad` in the class and train the model with this optimizer in `50` epochs. You can update the corresponding function in the `MyFNN` class.

[2.5 points]

In [22]:
```
myFFN = MyFFN(input_size = 28*28, num_classes = 10, hidden_sizes = [100, 100],
act = torch.nn.ReLU)
myFFN.create_FFN()
num_epochs = 50
for epoch in range(num_epochs):
    for i, (x, y) in enumerate(train_loader):
      x, y = x.to(device), y.to(device)
      myFFN.update_AdaGrad(x, y, learning_rate = 0.01)
    train_acc = compute_acc(myFFN, train_loader)
    train_loss = compute_loss(myFFN, train_loader)
    test_acc = compute_acc(myFFN, test_loader)
    test_loss = compute_loss(myFFN, test_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f}, Train
```

```
Acc: {train_acc*100:.2f}%, Test Loss: {test_loss:.4f}, Test Acc:
{test_acc*100:.2f}%")
```

```
Epoch 1/50, Train Loss: 0.2455, Train Acc: 92.86%, Test Loss: 0.2411, Test Acc:
92.95%
Epoch 2/50, Train Loss: 0.1960, Train Acc: 94.12%, Test Loss: 0.1951, Test Acc:
94.09%
Epoch 3/50, Train Loss: 0.1742, Train Acc: 94.77%, Test Loss: 0.1741, Test Acc:
94.60%
Epoch 4/50, Train Loss: 0.1583, Train Acc: 95.35%, Test Loss: 0.1654, Test Acc:
94.89%
Epoch 5/50, Train Loss: 0.1433, Train Acc: 95.91%, Test Loss: 0.1501, Test Acc:
95.44%
Epoch 6/50, Train Loss: 0.1308, Train Acc: 96.11%, Test Loss: 0.1384, Test Acc:
95.66%
Epoch 7/50, Train Loss: 0.1230, Train Acc: 96.36%, Test Loss: 0.1355, Test Acc:
95.77%
Epoch 8/50, Train Loss: 0.1114, Train Acc: 96.69%, Test Loss: 0.1233, Test Acc:
96.17%
Epoch 9/50, Train Loss: 0.1097, Train Acc: 96.74%, Test Loss: 0.1237, Test Acc:
96.20%
Epoch 10/50, Train Loss: 0.1052, Train Acc: 96.84%, Test Loss: 0.1212, Test Acc:
96.12%
Epoch 11/50, Train Loss: 0.1006, Train Acc: 97.06%, Test Loss: 0.1147, Test Acc:
96.45%
Epoch 12/50, Train Loss: 0.0960, Train Acc: 97.21%, Test Loss: 0.1130, Test Acc:
96.43%
Epoch 13/50, Train Loss: 0.0947, Train Acc: 97.19%, Test Loss: 0.1135, Test Acc:
96.35%
Epoch 14/50, Train Loss: 0.0893, Train Acc: 97.39%, Test Loss: 0.1086, Test Acc:
96.63%
Epoch 15/50, Train Loss: 0.0877, Train Acc: 97.41%, Test Loss: 0.1078, Test Acc:
96.48%
Epoch 16/50, Train Loss: 0.0827, Train Acc: 97.64%, Test Loss: 0.1040, Test Acc:
96.79%
Epoch 17/50, Train Loss: 0.0814, Train Acc: 97.58%, Test Loss: 0.1036, Test Acc:
96.73%
Epoch 18/50, Train Loss: 0.0801, Train Acc: 97.69%, Test Loss: 0.1024, Test Acc:
96.92%
Epoch 19/50, Train Loss: 0.0776, Train Acc: 97.75%, Test Loss: 0.1009, Test Acc:
96.87%
Epoch 20/50, Train Loss: 0.0752, Train Acc: 97.86%, Test Loss: 0.0995, Test Acc:
97.01%
Epoch 21/50, Train Loss: 0.0737, Train Acc: 97.94%, Test Loss: 0.0987, Test Acc:
97.05%
Epoch 22/50, Train Loss: 0.0715, Train Acc: 97.94%, Test Loss: 0.0964, Test Acc:
97.00%
Epoch 23/50, Train Loss: 0.0711, Train Acc: 97.95%, Test Loss: 0.0978, Test Acc:
96.96%
Epoch 24/50, Train Loss: 0.0704, Train Acc: 97.90%, Test Loss: 0.0974, Test Acc:
97.01%
Epoch 25/50, Train Loss: 0.0668, Train Acc: 98.10%, Test Loss: 0.0940, Test Acc:
97.17%
Epoch 26/50, Train Loss: 0.0661, Train Acc: 98.13%, Test Loss: 0.0946, Test Acc:
97.18%
Epoch 27/50, Train Loss: 0.0637, Train Acc: 98.17%, Test Loss: 0.0926, Test Acc:
97.18%
Epoch 28/50, Train Loss: 0.0653, Train Acc: 98.07%, Test Loss: 0.0944, Test Acc:
97.17%
Epoch 29/50, Train Loss: 0.0659, Train Acc: 98.06%, Test Loss: 0.0947, Test Acc:
97.07%
Epoch 30/50, Train Loss: 0.0617, Train Acc: 98.21%, Test Loss: 0.0930, Test Acc:
97.21%
```

```
Epoch 31/50, Train Loss: 0.0596, Train Acc: 98.31%, Test Loss: 0.0906, Test Acc:
97.26%
Epoch 32/50, Train Loss: 0.0603, Train Acc: 98.25%, Test Loss: 0.0915, Test Acc:
97.22%
Epoch 33/50, Train Loss: 0.0587, Train Acc: 98.33%, Test Loss: 0.0916, Test Acc:
97.16%
Epoch 34/50, Train Loss: 0.0573, Train Acc: 98.34%, Test Loss: 0.0902, Test Acc:
97.30%
Epoch 35/50, Train Loss: 0.0553, Train Acc: 98.44%, Test Loss: 0.0891, Test Acc:
97.28%
Epoch 36/50, Train Loss: 0.0564, Train Acc: 98.42%, Test Loss: 0.0905, Test Acc:
97.31%
Epoch 37/50, Train Loss: 0.0535, Train Acc: 98.49%, Test Loss: 0.0880, Test Acc:
97.37%
Epoch 38/50, Train Loss: 0.0529, Train Acc: 98.52%, Test Loss: 0.0881, Test Acc:
97.33%
Epoch 39/50, Train Loss: 0.0524, Train Acc: 98.52%, Test Loss: 0.0889, Test Acc:
97.28%
Epoch 40/50, Train Loss: 0.0549, Train Acc: 98.44%, Test Loss: 0.0927, Test Acc:
97.13%
Epoch 41/50, Train Loss: 0.0506, Train Acc: 98.59%, Test Loss: 0.0875, Test Acc:
97.31%
Epoch 42/50, Train Loss: 0.0501, Train Acc: 98.63%, Test Loss: 0.0877, Test Acc:
97.35%
Epoch 43/50, Train Loss: 0.0506, Train Acc: 98.53%, Test Loss: 0.0897, Test Acc:
97.31%
Epoch 44/50, Train Loss: 0.0493, Train Acc: 98.60%, Test Loss: 0.0875, Test Acc:
97.35%
Epoch 45/50, Train Loss: 0.0487, Train Acc: 98.62%, Test Loss: 0.0877, Test Acc:
97.34%
Epoch 46/50, Train Loss: 0.0481, Train Acc: 98.64%, Test Loss: 0.0879, Test Acc:
97.44%
Epoch 47/50, Train Loss: 0.0466, Train Acc: 98.73%, Test Loss: 0.0861, Test Acc:
97.38%
Epoch 48/50, Train Loss: 0.0466, Train Acc: 98.71%, Test Loss: 0.0876, Test Acc:
97.40%
Epoch 49/50, Train Loss: 0.0464, Train Acc: 98.69%, Test Loss: 0.0861, Test Acc:
97.40%
Epoch 50/50, Train Loss: 0.0450, Train Acc: 98.78%, Test Loss: 0.0855, Test Acc:
97.44%
```

Part 2: Deep Neural Networks (DNN)

[Total marks for this part: 25 points]

The second part of this assignment is to demonstrate your basis knowledge in deep learning that you have acquired from the lectures and tutorials materials. Most of the contents in this assignment are drawn from **the tutorials covered from weeks 1 to 2**. Going through these materials before attempting this assignment is highly recommended.

In the second part of this assignment, you are going to work with the FashionMNIST dataset for the image recognition task. It has the exact same format as MNIST (70,000 grayscale images of 28 × 28 pixels each with 10 classes), but the images represent fashion items rather than handwritten digits, so each class is more diverse, and the problem is significantly more challenging than MNIST.

In [25]:
```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

torch.manual_seed(1234)
```

Out[25]:
```
<torch._C.Generator at 0x1ebeefac750>
```

Load the Fashion MNIST using `torchvision`

In [26]:
```
transform = transforms.Compose([transforms.ToTensor(),
transforms.Normalize((0.5,), (0.5,))])

train_dataset_orgin = datasets.FashionMNIST(root='./data', train=True,
download=True, transform=transform)
test_dataset = datasets.FashionMNIST(root='./data', train=False, download=True,
transform=transform)

print(train_dataset_orgin.data.shape, train_dataset_orgin.targets.shape)
print(test_dataset.data.shape, test_dataset.targets.shape)

# Flatten the data
train_dataset_orgin.data = train_dataset_orgin.data.reshape(-1, 28*28)
test_dataset.data = test_dataset.data.reshape(-1, 28*28)

print(train_dataset_orgin.data.shape, train_dataset_orgin.targets.shape)
print(test_dataset.data.shape, test_dataset.targets.shape)

N = len(train_dataset_orgin)
print(f"Number of training samples: {N}")
N_train = int(0.9*N)
N_val = N - N_train
print(f"Number of training samples: {N_train}")
print(f"Number of validation samples: {N_val}")

train_dataset, val_dataset = torch.utils.data.random_split(train_dataset_orgin,
[N_train, N_val])


print(len(train_dataset))
print(len(val_dataset))


train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(dataset=val_dataset, batch_size=64, shuffle=False)
test_loader = DataLoader(dataset=test_dataset, batch_size=1000, shuffle=False)
```

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
torch.Size([60000, 784]) torch.Size([60000])
torch.Size([10000, 784]) torch.Size([10000])
Number of training samples: 60000
Number of training samples: 54000
Number of validation samples: 6000
54000
6000
```

Question 2.1: Write the code to visualize a mini-batch in `train_loader` including its images and labels.

[5 points]

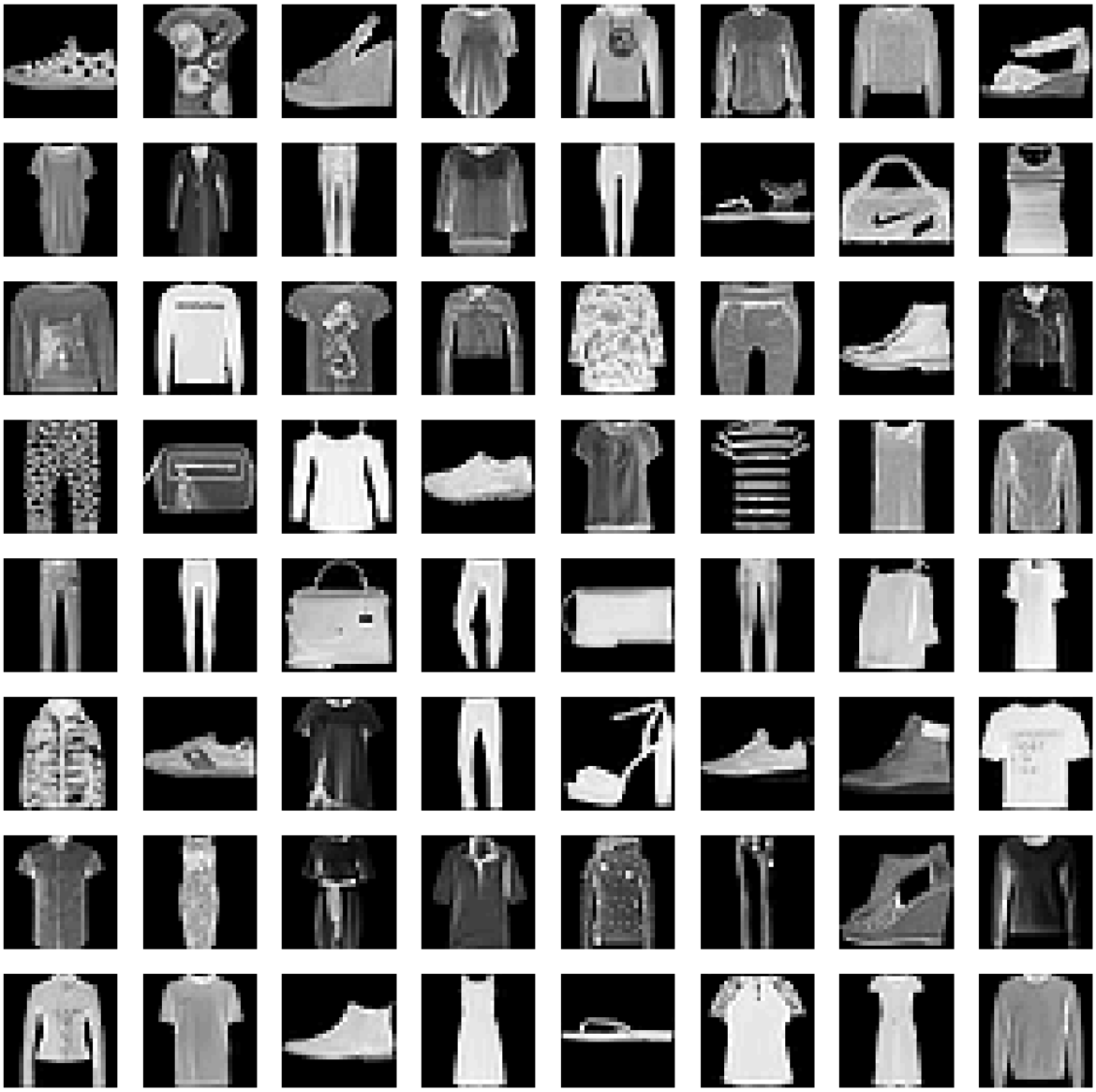
In [27]:
```
#Your code here
import math
import numpy as np
import matplotlib.pyplot as plt
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" #to run on window machine

fashion_class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                       'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

def imshow(img):
    img = img / 2 + 0.5  # unnormalize
    npimg = img.numpy().reshape(28, 28)  # flat 784-vector -> 28x28 image
    plt.imshow(npimg, cmap='gray')

def visualize_data(images, labels, images_per_row=8):
    n_images = len(images)
    n_rows = math.ceil(n_images / images_per_row)
    fig = plt.figure(figsize=(1.5*images_per_row, 1.5*n_rows))
    fig.patch.set_facecolor('white')
    for i in range(n_images):
        plt.subplot(n_rows, images_per_row, i+1)
        plt.xticks([])
        plt.yticks([])
        imshow(images[i])
        plt.xlabel(fashion_class_names[labels[i]])
    plt.show()

images, labels = next(iter(train_loader))
visualize_data(images, labels)
```



Question 2.2: Write the code for the feed-forward neural net using PyTorch

[5 points]

We now develop a feed-forward neural network with the architecture $784 \rightarrow 40(ReLU) \rightarrow 30(ReLU) \rightarrow 10(softmax)$. You can choose your own way to implement your network and an optimizer of interest. You should train model in 50 epochs and evaluate the trained model on the test set.

In [29]:
```
#Your code here
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class FFN(torch.nn.Module):
  def __init__(self, input_size=28*28, n1=40, n2=30, num_classes=10,
act=torch.nn.ReLU):
    super().__init__()
    self.net = torch.nn.Sequential(
        torch.nn.Linear(input_size, n1),
        act(),
        torch.nn.Linear(n1, n2),
        act(),
```

```
        torch.nn.Linear(n2, num_classes),
    )

  def forward(self, x):
    x = x.view(x.size(0), -1)
    return self.net(x)


def compute_loss(model, loss_fn, loader):
  model.eval()
  loss = 0.0
  with torch.no_grad():
    for batchX, batchY in loader:
      batchX, batchY = batchX.to(device).float(), batchY.to(device).long()
      loss += loss_fn(model(batchX), batchY).item()
  model.train()
  return loss / len(loader)


def compute_acc(model, loader):
  model.eval()
  correct, total = 0, 0
  with torch.no_grad():
    for batchX, batchY in loader:
      batchX, batchY = batchX.to(device).float(), batchY.to(device).long()
      preds = torch.argmax(model(batchX), dim=1)
      correct += (preds == batchY).sum().item()
      total += batchY.size(0)
  model.train()
  return correct / total


ffn_model = FFN(input_size=28*28, n1=40, n2=30, num_classes=10,
act=torch.nn.ReLU).to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(ffn_model.parameters(), lr=1e-3)

num_epochs = 10
for epoch in range(num_epochs):
    for batchX, batchY in train_loader:
        batchX, batchY = batchX.to(device).float(), batchY.to(device).long()
        optimizer.zero_grad()
        loss = loss_fn(ffn_model(batchX), batchY)
        loss.backward()
        optimizer.step()
    train_loss = compute_loss(ffn_model, loss_fn, train_loader)
    train_acc = compute_acc(ffn_model, train_loader)
    val_loss = compute_loss(ffn_model, loss_fn, val_loader)
    val_acc = compute_acc(ffn_model, val_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f}, Train
Acc: {train_acc*100:.2f}%, Val Loss: {val_loss:.4f}, Val Acc:
{val_acc*100:.2f}%")

test_loss = compute_loss(ffn_model, loss_fn, test_loader)
test_acc = compute_acc(ffn_model, test_loader)
print(f"Test Loss: {test_loss:.4f}, Test Acc: {test_acc*100:.2f}%")
```

```
Epoch 1/10, Train Loss: 0.4147, Train Acc: 85.22%, Val Loss: 0.4540, Val Acc:
83.32%
Epoch 2/10, Train Loss: 0.3757, Train Acc: 86.54%, Val Loss: 0.4236, Val Acc:
84.65%
Epoch 3/10, Train Loss: 0.3434, Train Acc: 87.43%, Val Loss: 0.3980, Val Acc:
85.58%
Epoch 4/10, Train Loss: 0.3273, Train Acc: 88.07%, Val Loss: 0.3772, Val Acc:
86.08%
Epoch 5/10, Train Loss: 0.3126, Train Acc: 88.46%, Val Loss: 0.3728, Val Acc:
86.58%
Epoch 6/10, Train Loss: 0.2928, Train Acc: 89.26%, Val Loss: 0.3593, Val Acc:
86.77%
Epoch 7/10, Train Loss: 0.3095, Train Acc: 88.44%, Val Loss: 0.3856, Val Acc:
86.48%
Epoch 8/10, Train Loss: 0.2917, Train Acc: 89.17%, Val Loss: 0.3687, Val Acc:
86.60%
Epoch 9/10, Train Loss: 0.2906, Train Acc: 89.25%, Val Loss: 0.3751, Val Acc:
86.55%
Epoch 10/10, Train Loss: 0.2706, Train Acc: 90.10%, Val Loss: 0.3679, Val Acc:
87.23%
Test Loss: 0.3743, Test Acc: 86.98%
```

Question 2.3: Tuning hyper-parameters with grid search

[5 points]

Assume that you need to tune the number of neurons on the first and second hidden layers $n_1 \in \{20, 40\}$, $n_2 \in \{20, 40\}$ and the used activation function $act \in \{sigmoid, tanh, relu\}$. The network has the architecture pattern $784 \rightarrow n_1(act) \rightarrow n_2(act) \rightarrow 10(softmax)$ where n_1, n_2, and act are in their grides. Write the code to tune the hyper-parameters n_1, n_2, and act. Note that you can freely choose the optimizer and learning rate of interest for this task.

In [27]:
```
#Your code here
grid_n1 = [20, 40]
grid_n2 = [20, 40]
grid_act = {'sigmoid': torch.nn.Sigmoid, 'tanh': torch.nn.Tanh, 'relu':
torch.nn.ReLU}

# fewer epochs per config to keep the 12-combo grid search tractable
grid_search_epochs = 15
grid_results = []

for n1 in grid_n1:
    for n2 in grid_n2:
        for act_name, act_fn in grid_act.items():
            torch.manual_seed(1234)
            model = FFN(input_size=28*28, n1=n1, n2=n2, num_classes=10,
act=act_fn).to(device)
            loss_fn = torch.nn.CrossEntropyLoss()
            optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

            for epoch in range(grid_search_epochs):
                for batchX, batchY in train_loader:
                    batchX, batchY = batchX.to(device).float(),
batchY.to(device).long()
                    optimizer.zero_grad()
                    loss = loss_fn(model(batchX), batchY)
```

```
                    loss.backward()
                    optimizer.step()

            val_loss = compute_loss(model, loss_fn, val_loader)
            val_acc = compute_acc(model, val_loader)
            grid_results.append({'n1': n1, 'n2': n2, 'act': act_name,
'val_loss': val_loss, 'val_acc': val_acc})
            print(f"n1={n1}, n2={n2}, act={act_name}: Val Loss={val_loss:.4f},
Val Acc={val_acc*100:.2f}%")

best = max(grid_results, key=lambda r: r['val_acc'])
print(f"\nBest config: n1={best['n1']}, n2={best['n2']}, act={best['act']}, Val
Acc={best['val_acc']*100:.2f}%")
```

```
n1=20, n2=20, act=sigmoid: Val Loss=0.3994, Val Acc=85.82%
n1=20, n2=20, act=tanh: Val Loss=0.3973, Val Acc=86.05%
n1=20, n2=20, act=relu: Val Loss=0.3845, Val Acc=86.40%
n1=20, n2=40, act=sigmoid: Val Loss=0.4146, Val Acc=85.53%
n1=20, n2=40, act=tanh: Val Loss=0.4070, Val Acc=85.85%
n1=20, n2=40, act=relu: Val Loss=0.3766, Val Acc=86.75%
n1=40, n2=20, act=sigmoid: Val Loss=0.3652, Val Acc=87.37%
n1=40, n2=20, act=tanh: Val Loss=0.3687, Val Acc=87.35%
n1=40, n2=20, act=relu: Val Loss=0.3601, Val Acc=87.22%
n1=40, n2=40, act=sigmoid: Val Loss=0.3597, Val Acc=87.05%
n1=40, n2=40, act=tanh: Val Loss=0.3565, Val Acc=87.23%
n1=40, n2=40, act=relu: Val Loss=0.3443, Val Acc=87.72%

Best config: n1=40, n2=40, act=relu, Val Acc=87.72%
```

Question 2.4: Implement the loss with the form: $loss(p, y) = CE(1_y, p) + \lambda H(p)$ where $H(p) = -\sum_{i=1}^{M} p_i \log p_i$ is the entropy of p, p is the prediction probabilities of a data point x with the ground-truth label y, 1_y is an one-hot label, and $\lambda > 0$ is a trade-off parameter. Set $\lambda = 0.1$ to train a model.

[5 points]

In [32]:
```
#Your code here
# Question 2.4:  loss(p, y) = CE(1_y, p) + lambda * H(p)   with   H(p) = -sum_i
p_i log p_i

class EntropyRegularisedCE(torch.nn.Module):
    """
    loss(p, y) = CE(1_y, p) + lambda * H(p)

    p = softmax(h) are the prediction probabilities of a data point with logits
h,
    1_y is the one-hot ground-truth label, so CE(1_y,p) = -log p_y, and
    H(p) = -sum_{i=1}^{M} p_i log p_i is the entropy of p.
    """
    def __init__(self, lam=0.1):
        super().__init__()
        self.lam = lam

    def forward(self, logits, y):
        log_p = torch.nn.functional.log_softmax(logits, dim=1)   # log p_i
(numerically stable softmax + log)
        p = torch.nn.functional.softmax(logits, dim=1)
        ce = -log_p.gather(dim=1, index=y.unsqueeze(1)).squeeze(1)  # CE(1_y,
```

```
p) = -log p_y  (one value per example)
        entropy = -(p * log_p).sum(dim=1)                           # H(p) = -
sum_i p_i log p_i  (one value per example)
        return (ce + self.lam * entropy).mean()


def train_one_epoch(model, loss_fn, optimizer, loader):
    """One pass over the training set with a standard optimizer step."""
    model.train()
    for batchX, batchY in loader:
        batchX, batchY = batchX.to(device).float(), batchY.to(device).long()
        optimizer.zero_grad()
        loss = loss_fn(model(batchX), batchY)
        loss.backward()
        optimizer.step()


def fit_ffn(model, loss_fn, optimizer, num_epochs=10,
train_step=train_one_epoch):
    """
    Train `model` with `loss_fn` and `optimizer` for `num_epochs` epochs and
report the train/val
    metrics after each epoch. `train_step` is the function used to train one
epoch
    (it is replaced by the SAM step in Question 2.5). Returns the history of the
metrics.
    """
    ce_loss = torch.nn.CrossEntropyLoss()
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
    for epoch in range(num_epochs):
        train_step(model, loss_fn, optimizer, train_loader)
        train_loss = compute_loss(model, ce_loss, train_loader)   # reported
with the plain CE so that runs are comparable
        train_acc = compute_acc(model, train_loader)
        val_loss = compute_loss(model, ce_loss, val_loader)
        val_acc = compute_acc(model, val_loader)
        for k, v in zip(history, (train_loss, train_acc, val_loss, val_acc)):
            history[k].append(v)
        print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f},
Train Acc: {train_acc*100:.2f}%, "
              f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc*100:.2f}%")
    return history


# Train the 784 -> 40 (ReLU) -> 30 (ReLU) -> 10 network of Question 2.2 with the
new loss (lambda = 0.1)
torch.manual_seed(1234)
lam = 0.1
entropy_loss_fn = EntropyRegularisedCE(lam=lam)
ffn_entropy = FFN(input_size=28*28, n1=40, n2=30, num_classes=10,
act=torch.nn.ReLU).to(device)
optimizer = torch.optim.Adam(ffn_entropy.parameters(), lr=1e-3)

num_epochs = 10
history_entropy = fit_ffn(ffn_entropy, entropy_loss_fn, optimizer,
num_epochs=num_epochs)

test_loss_entropy = compute_loss(ffn_entropy, torch.nn.CrossEntropyLoss(),
test_loader)
test_acc_entropy = compute_acc(ffn_entropy, test_loader)
```

```
print(f"\n[CE + {lam} * H(p)]  Test CE Loss: {test_loss_entropy:.4f}, Test Acc:
{test_acc_entropy*100:.2f}%")
```

```
Epoch 1/50, Train Loss: 0.4271, Train Acc: 84.82%, Val Loss: 0.4699, Val Acc:
83.15%
Epoch 2/50, Train Loss: 0.3757, Train Acc: 86.43%, Val Loss: 0.4197, Val Acc:
84.90%
Epoch 3/50, Train Loss: 0.3514, Train Acc: 87.25%, Val Loss: 0.4045, Val Acc:
85.53%
Epoch 4/50, Train Loss: 0.3280, Train Acc: 88.00%, Val Loss: 0.3819, Val Acc:
86.17%
Epoch 5/50, Train Loss: 0.3165, Train Acc: 88.37%, Val Loss: 0.3793, Val Acc:
86.48%
Epoch 6/50, Train Loss: 0.3008, Train Acc: 88.85%, Val Loss: 0.3635, Val Acc:
87.20%
Epoch 7/50, Train Loss: 0.2981, Train Acc: 88.97%, Val Loss: 0.3692, Val Acc:
86.88%
Epoch 8/50, Train Loss: 0.2806, Train Acc: 89.68%, Val Loss: 0.3569, Val Acc:
87.67%
Epoch 9/50, Train Loss: 0.2840, Train Acc: 89.44%, Val Loss: 0.3721, Val Acc:
86.87%
Epoch 10/50, Train Loss: 0.2754, Train Acc: 89.83%, Val Loss: 0.3681, Val Acc:
87.48%
Epoch 11/50, Train Loss: 0.2633, Train Acc: 90.21%, Val Loss: 0.3646, Val Acc:
87.47%
Epoch 12/50, Train Loss: 0.2488, Train Acc: 90.71%, Val Loss: 0.3552, Val Acc:
87.43%
Epoch 13/50, Train Loss: 0.2481, Train Acc: 90.85%, Val Loss: 0.3675, Val Acc:
87.33%
Epoch 14/50, Train Loss: 0.2414, Train Acc: 91.08%, Val Loss: 0.3704, Val Acc:
87.53%
Epoch 15/50, Train Loss: 0.2517, Train Acc: 90.56%, Val Loss: 0.3701, Val Acc:
87.62%
Epoch 16/50, Train Loss: 0.2408, Train Acc: 91.13%, Val Loss: 0.3756, Val Acc:
87.57%
Epoch 17/50, Train Loss: 0.2335, Train Acc: 91.49%, Val Loss: 0.3787, Val Acc:
87.47%
Epoch 18/50, Train Loss: 0.2376, Train Acc: 91.11%, Val Loss: 0.3737, Val Acc:
87.58%
Epoch 19/50, Train Loss: 0.2251, Train Acc: 91.58%, Val Loss: 0.3710, Val Acc:
87.62%
Epoch 20/50, Train Loss: 0.2232, Train Acc: 91.64%, Val Loss: 0.3799, Val Acc:
87.55%
Epoch 21/50, Train Loss: 0.2259, Train Acc: 91.56%, Val Loss: 0.3922, Val Acc:
87.60%
Epoch 22/50, Train Loss: 0.2317, Train Acc: 91.18%, Val Loss: 0.3899, Val Acc:
86.73%
Epoch 23/50, Train Loss: 0.2009, Train Acc: 92.50%, Val Loss: 0.3786, Val Acc:
87.93%
Epoch 24/50, Train Loss: 0.1993, Train Acc: 92.73%, Val Loss: 0.3772, Val Acc:
87.95%
Epoch 25/50, Train Loss: 0.1988, Train Acc: 92.61%, Val Loss: 0.3731, Val Acc:
88.13%
Epoch 26/50, Train Loss: 0.2047, Train Acc: 92.44%, Val Loss: 0.3892, Val Acc:
87.52%
Epoch 27/50, Train Loss: 0.1964, Train Acc: 92.74%, Val Loss: 0.3990, Val Acc:
87.78%
Epoch 28/50, Train Loss: 0.1962, Train Acc: 92.76%, Val Loss: 0.3886, Val Acc:
87.47%
Epoch 29/50, Train Loss: 0.1892, Train Acc: 92.93%, Val Loss: 0.4037, Val Acc:
87.77%
Epoch 30/50, Train Loss: 0.1863, Train Acc: 93.07%, Val Loss: 0.4091, Val Acc:
87.78%
```

```
Epoch 31/50, Train Loss: 0.2004, Train Acc: 92.48%, Val Loss: 0.4189, Val Acc:
87.22%
Epoch 32/50, Train Loss: 0.1815, Train Acc: 93.36%, Val Loss: 0.4113, Val Acc:
87.90%
Epoch 33/50, Train Loss: 0.1846, Train Acc: 93.17%, Val Loss: 0.4072, Val Acc:
88.05%
Epoch 34/50, Train Loss: 0.1715, Train Acc: 93.65%, Val Loss: 0.4106, Val Acc:
87.57%
Epoch 35/50, Train Loss: 0.1696, Train Acc: 93.82%, Val Loss: 0.4180, Val Acc:
87.98%
Epoch 36/50, Train Loss: 0.1739, Train Acc: 93.54%, Val Loss: 0.4118, Val Acc:
88.13%
Epoch 37/50, Train Loss: 0.1831, Train Acc: 93.23%, Val Loss: 0.4252, Val Acc:
87.78%
Epoch 38/50, Train Loss: 0.1755, Train Acc: 93.50%, Val Loss: 0.4285, Val Acc:
87.87%
Epoch 39/50, Train Loss: 0.1743, Train Acc: 93.47%, Val Loss: 0.4208, Val Acc:
88.13%
Epoch 40/50, Train Loss: 0.1693, Train Acc: 93.71%, Val Loss: 0.4390, Val Acc:
88.00%
Epoch 41/50, Train Loss: 0.1624, Train Acc: 94.05%, Val Loss: 0.4338, Val Acc:
88.10%
Epoch 42/50, Train Loss: 0.1553, Train Acc: 94.26%, Val Loss: 0.4354, Val Acc:
87.85%
Epoch 43/50, Train Loss: 0.1597, Train Acc: 94.04%, Val Loss: 0.4497, Val Acc:
87.98%
Epoch 44/50, Train Loss: 0.1869, Train Acc: 92.98%, Val Loss: 0.4621, Val Acc:
86.95%
Epoch 45/50, Train Loss: 0.1683, Train Acc: 93.86%, Val Loss: 0.4714, Val Acc:
87.57%
Epoch 46/50, Train Loss: 0.1607, Train Acc: 94.11%, Val Loss: 0.4640, Val Acc:
87.47%
Epoch 47/50, Train Loss: 0.1505, Train Acc: 94.54%, Val Loss: 0.4400, Val Acc:
88.10%
Epoch 48/50, Train Loss: 0.1626, Train Acc: 93.96%, Val Loss: 0.4756, Val Acc:
87.70%
Epoch 49/50, Train Loss: 0.1492, Train Acc: 94.52%, Val Loss: 0.4896, Val Acc:
87.45%
Epoch 50/50, Train Loss: 0.1413, Train Acc: 94.84%, Val Loss: 0.4717, Val Acc:
88.02%

[CE + 0.1 * H(p)]  Test CE Loss: 0.4769, Test Acc: 87.59%
[plain CE, Q2.2]   Test CE Loss: 0.3743, Test Acc: 86.98%
```

Question 2.5: Experimenting with **sharpness-aware minimization** technique

[5 points]

Sharpness-aware minimization (SAM) (i.e., link for main paper from Google Deepmind) is a simple yet but efficient technique to improve the generalization ability of deep learning models on unseen data examples. In your research or your work, you might potentially use this idea. Your task is to read the paper and implement *Sharpness-aware minimization (SAM)*. Finally, you need to apply SAM to the best architecture found in **Question 2.3**.

In [29]:
```python
#Your code here

class SAM(torch.optim.Optimizer):
    """
```

```
    SAM wraps a base optimizer (e.g. Adam). Usage per mini-batch:
        loss_fn(model(x), y).backward();  sam.first_step(zero_grad=True)  steps 1-2
        loss_fn(model(x), y).backward();  sam.second_step(zero_grad=True) steps 3-4
    """
    def __init__(self, params, base_optimizer, rho=0.05, **kwargs):
        assert rho >= 0.0, "rho must be non-negative"
        defaults = dict(rho=rho, **kwargs)
        super().__init__(params, defaults)
        self.base_optimizer = base_optimizer(self.param_groups, **kwargs)
        self.param_groups = self.base_optimizer.param_groups   # share the parameter groups with the base optimizer

    @torch.no_grad()
    def first_step(self, zero_grad=False):
        """Climb to the adversarial point w + eps_hat (Eq. 2). The gradient of L_B(w) must have been computed."""
        grad_norm = self._grad_norm()
        for group in self.param_groups:
            scale = group["rho"] / (grad_norm + 1e-12)
            for p in group["params"]:
                if p.grad is None:
                    continue
                e_w = p.grad * scale                     # eps_hat = rho * g / ||g||
                p.add_(e_w)                              # w  ->  w + eps_hat
                self.state[p]["e_w"] = e_w
        if zero_grad:
            self.zero_grad()

    @torch.no_grad()
    def second_step(self, zero_grad=False):
        """Go back to w and update it with the gradient computed at w + eps_hat (Eq. 3)."""
        for group in self.param_groups:
            for p in group["params"]:
                if p.grad is None or "e_w" not in self.state[p]:
                    continue
                p.sub_(self.state[p]["e_w"])             # w + eps_hat  ->  w
        self.base_optimizer.step()                       # w <- w - eta * g_sam (base optimizer update rule)
        if zero_grad:
            self.zero_grad()

    def _grad_norm(self):
        """||grad L_B(w)||_2 over all parameters."""
        norms = [p.grad.norm(p=2) for group in self.param_groups for p in group["params"] if p.grad is not None]
        return torch.norm(torch.stack(norms), p=2)

    def step(self, closure=None):
        raise RuntimeError("Use first_step() and second_step() instead of step().")


def sam_train_one_epoch(model, loss_fn, optimizer, loader):
    """One pass over the training set with the two-pass SAM update (Algorithm 1 of the paper)."""
    model.train()
```

```
    for batchX, batchY in loader:
        batchX, batchY = batchX.to(device).float(), batchY.to(device).long()
        # 1st pass: gradient at w, then climb to w + eps_hat
        loss_fn(model(batchX), batchY).backward()
        optimizer.first_step(zero_grad=True)
        # 2nd pass: gradient at w + eps_hat, then go back to w and update with
the base optimizer
        loss_fn(model(batchX), batchY).backward()
        optimizer.second_step(zero_grad=True)


# Apply SAM to the best architecture found in Question 2.3
best_n1, best_n2, best_act_name = best['n1'], best['n2'], best['act']
best_act = grid_act[best_act_name]
print(f"Best architecture from Q2.3: 784 -> {best_n1} ({best_act_name}) ->
{best_n2} ({best_act_name}) -> 10\n")

num_epochs = 50
rho = 0.05                      # neighbourhood size (the value used for most
experiments in the paper)
loss_fn = torch.nn.CrossEntropyLoss()

# (i) best architecture trained with SAM (base optimizer: Adam, lr = 1e-3, as in
Q2.3)
torch.manual_seed(1234)
ffn_sam = FFN(input_size=28*28, n1=best_n1, n2=best_n2, num_classes=10,
act=best_act).to(device)
sam_optimizer = SAM(ffn_sam.parameters(), torch.optim.Adam, rho=rho, lr=1e-3)
print("Training with SAM:")
history_sam = fit_ffn(ffn_sam, loss_fn, sam_optimizer, num_epochs=num_epochs,
train_step=sam_train_one_epoch)

# (ii) the same architecture trained for the same number of epochs WITHOUT SAM
(plain Adam) as the baseline
torch.manual_seed(1234)
ffn_base = FFN(input_size=28*28, n1=best_n1, n2=best_n2, num_classes=10,
act=best_act).to(device)
base_optimizer = torch.optim.Adam(ffn_base.parameters(), lr=1e-3)
print("\nTraining without SAM (baseline):")
history_base = fit_ffn(ffn_base, loss_fn, base_optimizer, num_epochs=num_epochs)

print("\nEvaluation on the test set")
for name, m in [("Adam + SAM", ffn_sam), ("Adam (no SAM)", ffn_base)]:
    print(f"{name:14s}: Test Loss: {compute_loss(m, loss_fn, test_loader):.4f},
Test Acc: {compute_acc(m, test_loader)*100:.2f}%")
```

```
Best architecture from Q2.3: 784 -> 40 (relu) -> 40 (relu) -> 10

Training with SAM:
Epoch 1/50, Train Loss: 0.4334, Train Acc: 84.36%, Val Loss: 0.4677, Val Acc:
82.72%
Epoch 2/50, Train Loss: 0.3763, Train Acc: 86.61%, Val Loss: 0.4161, Val Acc:
85.02%
Epoch 3/50, Train Loss: 0.3548, Train Acc: 86.97%, Val Loss: 0.4030, Val Acc:
84.92%
Epoch 4/50, Train Loss: 0.3310, Train Acc: 87.89%, Val Loss: 0.3827, Val Acc:
85.77%
Epoch 5/50, Train Loss: 0.3168, Train Acc: 88.59%, Val Loss: 0.3691, Val Acc:
86.67%
Epoch 6/50, Train Loss: 0.2944, Train Acc: 89.12%, Val Loss: 0.3626, Val Acc:
86.63%
Epoch 7/50, Train Loss: 0.2977, Train Acc: 89.28%, Val Loss: 0.3612, Val Acc:
86.50%
Epoch 8/50, Train Loss: 0.2813, Train Acc: 89.73%, Val Loss: 0.3540, Val Acc:
86.98%
Epoch 9/50, Train Loss: 0.2767, Train Acc: 89.93%, Val Loss: 0.3546, Val Acc:
86.85%
Epoch 10/50, Train Loss: 0.2661, Train Acc: 90.13%, Val Loss: 0.3481, Val Acc:
87.27%
Epoch 11/50, Train Loss: 0.2589, Train Acc: 90.74%, Val Loss: 0.3454, Val Acc:
87.20%
Epoch 12/50, Train Loss: 0.2615, Train Acc: 90.44%, Val Loss: 0.3480, Val Acc:
87.28%
Epoch 13/50, Train Loss: 0.2462, Train Acc: 91.02%, Val Loss: 0.3375, Val Acc:
87.92%
Epoch 14/50, Train Loss: 0.2480, Train Acc: 90.87%, Val Loss: 0.3430, Val Acc:
87.95%
Epoch 15/50, Train Loss: 0.2462, Train Acc: 90.92%, Val Loss: 0.3522, Val Acc:
86.98%
Epoch 16/50, Train Loss: 0.2347, Train Acc: 91.50%, Val Loss: 0.3389, Val Acc:
88.02%
Epoch 17/50, Train Loss: 0.2328, Train Acc: 91.49%, Val Loss: 0.3368, Val Acc:
87.82%
Epoch 18/50, Train Loss: 0.2334, Train Acc: 91.61%, Val Loss: 0.3390, Val Acc:
87.58%
Epoch 19/50, Train Loss: 0.2269, Train Acc: 91.68%, Val Loss: 0.3398, Val Acc:
88.07%
Epoch 20/50, Train Loss: 0.2318, Train Acc: 91.46%, Val Loss: 0.3490, Val Acc:
87.33%
Epoch 21/50, Train Loss: 0.2226, Train Acc: 91.77%, Val Loss: 0.3429, Val Acc:
88.13%
Epoch 22/50, Train Loss: 0.2408, Train Acc: 91.01%, Val Loss: 0.3682, Val Acc:
86.95%
Epoch 23/50, Train Loss: 0.2111, Train Acc: 92.37%, Val Loss: 0.3414, Val Acc:
87.83%
Epoch 24/50, Train Loss: 0.2144, Train Acc: 92.27%, Val Loss: 0.3523, Val Acc:
87.62%
Epoch 25/50, Train Loss: 0.2107, Train Acc: 92.40%, Val Loss: 0.3456, Val Acc:
87.68%
Epoch 26/50, Train Loss: 0.2007, Train Acc: 92.57%, Val Loss: 0.3461, Val Acc:
88.13%
Epoch 27/50, Train Loss: 0.2121, Train Acc: 92.23%, Val Loss: 0.3557, Val Acc:
87.53%
Epoch 28/50, Train Loss: 0.1974, Train Acc: 92.78%, Val Loss: 0.3447, Val Acc:
87.92%
Epoch 29/50, Train Loss: 0.1949, Train Acc: 92.86%, Val Loss: 0.3432, Val Acc:
```

```
88.05%
Epoch 30/50, Train Loss: 0.1941, Train Acc: 92.73%, Val Loss: 0.3507, Val Acc:
87.88%
Epoch 31/50, Train Loss: 0.1958, Train Acc: 92.71%, Val Loss: 0.3598, Val Acc:
88.05%
Epoch 32/50, Train Loss: 0.1924, Train Acc: 92.97%, Val Loss: 0.3510, Val Acc:
87.87%
Epoch 33/50, Train Loss: 0.1952, Train Acc: 92.84%, Val Loss: 0.3614, Val Acc:
87.62%
Epoch 34/50, Train Loss: 0.1849, Train Acc: 93.27%, Val Loss: 0.3520, Val Acc:
88.28%
Epoch 35/50, Train Loss: 0.1853, Train Acc: 93.24%, Val Loss: 0.3589, Val Acc:
87.78%
Epoch 36/50, Train Loss: 0.1867, Train Acc: 93.08%, Val Loss: 0.3654, Val Acc:
87.97%
Epoch 37/50, Train Loss: 0.1815, Train Acc: 93.35%, Val Loss: 0.3713, Val Acc:
87.73%
Epoch 38/50, Train Loss: 0.1835, Train Acc: 93.23%, Val Loss: 0.3641, Val Acc:
87.90%
Epoch 39/50, Train Loss: 0.1819, Train Acc: 93.21%, Val Loss: 0.3711, Val Acc:
87.93%
Epoch 40/50, Train Loss: 0.1781, Train Acc: 93.55%, Val Loss: 0.3660, Val Acc:
87.90%
Epoch 41/50, Train Loss: 0.1749, Train Acc: 93.54%, Val Loss: 0.3678, Val Acc:
88.28%
Epoch 42/50, Train Loss: 0.1683, Train Acc: 93.79%, Val Loss: 0.3693, Val Acc:
87.83%
Epoch 43/50, Train Loss: 0.1703, Train Acc: 93.76%, Val Loss: 0.3637, Val Acc:
87.97%
Epoch 44/50, Train Loss: 0.1711, Train Acc: 93.78%, Val Loss: 0.3670, Val Acc:
87.85%
Epoch 45/50, Train Loss: 0.1651, Train Acc: 94.09%, Val Loss: 0.3611, Val Acc:
87.85%
Epoch 46/50, Train Loss: 0.1608, Train Acc: 94.15%, Val Loss: 0.3651, Val Acc:
88.28%
Epoch 47/50, Train Loss: 0.1725, Train Acc: 93.47%, Val Loss: 0.3901, Val Acc:
87.62%
Epoch 48/50, Train Loss: 0.1618, Train Acc: 94.10%, Val Loss: 0.3740, Val Acc:
88.03%
Epoch 49/50, Train Loss: 0.1579, Train Acc: 94.43%, Val Loss: 0.3740, Val Acc:
88.30%
Epoch 50/50, Train Loss: 0.1675, Train Acc: 93.81%, Val Loss: 0.3838, Val Acc:
87.72%

Training without SAM (baseline):
Epoch 1/50, Train Loss: 0.4296, Train Acc: 84.43%, Val Loss: 0.4650, Val Acc:
82.78%
Epoch 2/50, Train Loss: 0.3735, Train Acc: 86.63%, Val Loss: 0.4156, Val Acc:
85.12%
Epoch 3/50, Train Loss: 0.3659, Train Acc: 86.43%, Val Loss: 0.4234, Val Acc:
84.77%
Epoch 4/50, Train Loss: 0.3302, Train Acc: 87.91%, Val Loss: 0.3891, Val Acc:
85.70%
Epoch 5/50, Train Loss: 0.3274, Train Acc: 88.01%, Val Loss: 0.3822, Val Acc:
86.12%
Epoch 6/50, Train Loss: 0.3034, Train Acc: 88.71%, Val Loss: 0.3740, Val Acc:
86.38%
Epoch 7/50, Train Loss: 0.3073, Train Acc: 88.65%, Val Loss: 0.3750, Val Acc:
86.25%
Epoch 8/50, Train Loss: 0.2897, Train Acc: 89.14%, Val Loss: 0.3698, Val Acc:
```

```
86.33%
Epoch 9/50, Train Loss: 0.2695, Train Acc: 89.91%, Val Loss: 0.3490, Val Acc:
87.18%
Epoch 10/50, Train Loss: 0.2662, Train Acc: 90.12%, Val Loss: 0.3524, Val Acc:
87.07%
Epoch 11/50, Train Loss: 0.2609, Train Acc: 90.36%, Val Loss: 0.3565, Val Acc:
87.23%
Epoch 12/50, Train Loss: 0.2519, Train Acc: 90.65%, Val Loss: 0.3585, Val Acc:
87.37%
Epoch 13/50, Train Loss: 0.2388, Train Acc: 91.18%, Val Loss: 0.3485, Val Acc:
87.72%
Epoch 14/50, Train Loss: 0.2376, Train Acc: 91.27%, Val Loss: 0.3489, Val Acc:
87.98%
Epoch 15/50, Train Loss: 0.2254, Train Acc: 91.65%, Val Loss: 0.3473, Val Acc:
88.07%
Epoch 16/50, Train Loss: 0.2423, Train Acc: 90.96%, Val Loss: 0.3729, Val Acc:
86.95%
Epoch 17/50, Train Loss: 0.2208, Train Acc: 91.90%, Val Loss: 0.3501, Val Acc:
88.18%
Epoch 18/50, Train Loss: 0.2213, Train Acc: 91.86%, Val Loss: 0.3623, Val Acc:
87.80%
Epoch 19/50, Train Loss: 0.2221, Train Acc: 91.71%, Val Loss: 0.3601, Val Acc:
88.02%
Epoch 20/50, Train Loss: 0.2189, Train Acc: 91.96%, Val Loss: 0.3691, Val Acc:
87.62%
Epoch 21/50, Train Loss: 0.2163, Train Acc: 92.01%, Val Loss: 0.3778, Val Acc:
88.00%
Epoch 22/50, Train Loss: 0.2185, Train Acc: 91.95%, Val Loss: 0.3843, Val Acc:
87.37%
Epoch 23/50, Train Loss: 0.2023, Train Acc: 92.64%, Val Loss: 0.3715, Val Acc:
88.13%
Epoch 24/50, Train Loss: 0.1962, Train Acc: 92.74%, Val Loss: 0.3794, Val Acc:
87.30%
Epoch 25/50, Train Loss: 0.2044, Train Acc: 92.33%, Val Loss: 0.3845, Val Acc:
87.40%
Epoch 26/50, Train Loss: 0.1995, Train Acc: 92.60%, Val Loss: 0.3860, Val Acc:
87.72%
Epoch 27/50, Train Loss: 0.2065, Train Acc: 92.24%, Val Loss: 0.4014, Val Acc:
87.45%
Epoch 28/50, Train Loss: 0.1750, Train Acc: 93.54%, Val Loss: 0.3819, Val Acc:
87.73%
Epoch 29/50, Train Loss: 0.1921, Train Acc: 92.85%, Val Loss: 0.3982, Val Acc:
87.13%
Epoch 30/50, Train Loss: 0.1764, Train Acc: 93.42%, Val Loss: 0.3843, Val Acc:
88.27%
Epoch 31/50, Train Loss: 0.1751, Train Acc: 93.52%, Val Loss: 0.3929, Val Acc:
87.98%
Epoch 32/50, Train Loss: 0.1727, Train Acc: 93.67%, Val Loss: 0.4025, Val Acc:
87.67%
Epoch 33/50, Train Loss: 0.1906, Train Acc: 92.74%, Val Loss: 0.4196, Val Acc:
87.03%
Epoch 34/50, Train Loss: 0.1825, Train Acc: 93.25%, Val Loss: 0.4142, Val Acc:
87.68%
Epoch 35/50, Train Loss: 0.1710, Train Acc: 93.63%, Val Loss: 0.4101, Val Acc:
87.47%
Epoch 36/50, Train Loss: 0.1686, Train Acc: 93.75%, Val Loss: 0.4077, Val Acc:
87.35%
Epoch 37/50, Train Loss: 0.1760, Train Acc: 93.40%, Val Loss: 0.4385, Val Acc:
87.43%
Epoch 38/50, Train Loss: 0.1666, Train Acc: 93.91%, Val Loss: 0.4212, Val Acc:
```

```
87.77%
Epoch 39/50, Train Loss: 0.1746, Train Acc: 93.39%, Val Loss: 0.4367, Val Acc:
87.15%
Epoch 40/50, Train Loss: 0.1703, Train Acc: 93.69%, Val Loss: 0.4281, Val Acc:
87.53%
Epoch 41/50, Train Loss: 0.1659, Train Acc: 93.89%, Val Loss: 0.4291, Val Acc:
87.43%
Epoch 42/50, Train Loss: 0.1590, Train Acc: 94.06%, Val Loss: 0.4336, Val Acc:
87.38%
Epoch 43/50, Train Loss: 0.1498, Train Acc: 94.44%, Val Loss: 0.4346, Val Acc:
88.17%
Epoch 44/50, Train Loss: 0.1433, Train Acc: 94.80%, Val Loss: 0.4404, Val Acc:
87.68%
Epoch 45/50, Train Loss: 0.1464, Train Acc: 94.45%, Val Loss: 0.4279, Val Acc:
87.92%
Epoch 46/50, Train Loss: 0.1588, Train Acc: 94.01%, Val Loss: 0.4629, Val Acc:
86.85%
Epoch 47/50, Train Loss: 0.1539, Train Acc: 94.25%, Val Loss: 0.4458, Val Acc:
87.43%
Epoch 48/50, Train Loss: 0.1439, Train Acc: 94.64%, Val Loss: 0.4462, Val Acc:
87.45%
Epoch 49/50, Train Loss: 0.1519, Train Acc: 94.31%, Val Loss: 0.4663, Val Acc:
87.50%
Epoch 50/50, Train Loss: 0.1309, Train Acc: 95.27%, Val Loss: 0.4613, Val Acc:
87.68%

Evaluation on the test set
Adam + SAM    : Test Loss: 0.3950, Test Acc: 87.50%
Adam (no SAM) : Test Loss: 0.4511, Test Acc: 87.66%
```

Part 3: Convolutional Neural Networks and Image Classification

[Total marks for this part: 45 points]

The third part of this assignment is to demonstrate your basis knowledge in deep learning that you have acquired from the lectures and tutorials materials. Most of the contents in this assignment are drawn from **the tutorials covered from weeks 3 to 6**. Going through these materials before attempting this assignment is highly recommended.

The dataset used for this part is a specific dataset for this unit consisting of approximately $10,000$ images of 20 classes of Animals, each of which has approximately 500 images. You can download the dataset at download here if you want to do your assignment on your machine.

In [1]:
```python
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" #to run on window machine (same as
Part 2): avoids the OpenMP "Error #15" kernel crash on the first plot
import requests
import tarfile
import time
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
import torchvision.models as models
import torch.nn as nn
import torch
```

```
import PIL.Image
import pathlib
from torchsummary import summary
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np

# check if CUDA is available
train_on_gpu = torch.cuda.is_available()

if not train_on_gpu:
    print('CUDA is not available.  Training on CPU ...')
else:
    print('CUDA is available!  Training on GPU ...')

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(1234)
```

```
CUDA is available!  Training on GPU ...
```

Out[1]:
```
<torch._C.Generator at 0x1ebeefac750>
```

Download the dataset to the folder of this Google Colab.

In [31]:
```
!gdown --fuzzy
https://drive.google.com/file/d/10Y65ykpja1t6UWt3imPPbZkqhe0kOPL9/view?
usp=sharing # new url v1
#!gdown --fuzzy
https://drive.google.com/file/d/1uc7uQ5myz7k74ZO460Q04B35_GekQeGi/view?
usp=sharing # new url v2
```

```
'gdown' is not recognized as an internal or external command,
operable program or batch file.
```

We unzip the dataset to the folder.

In [32]:
```
!unzip -q Animals_Dataset_v1.zip
# !unzip -q Animals_Dataset_v2.zip
# !unzip -q Animals_Dataset.zip
```

```
'unzip' is not recognized as an internal or external command,
operable program or batch file.
```

In [2]:
```
data_dir = "data/Animals_Dataset_v1/FIT5215_Dataset"

# We resize the images to [3,64,64]
transform = transforms.Compose([transforms.Resize((64,64)),  #resises the image
so it can be perfect for our model.
                                transforms.RandomHorizontalFlip(), # FLips
the image w.r.t horizontal axis
                                #transforms.RandomRotation(4),
#Rotates the image to a specified angel
                                #transforms.RandomAffine(0, shear=10,
scale=(0.8,1.2)), #Performs actions like zooms, change shear angles.
                                transforms.ColorJitter(brightness=0.2,
contrast=0.2, saturation=0.2), # Set the color params
                                transforms.ToTensor(), # convert the image
to tensor so that it can work with torch
                                transforms.Normalize((0.5, 0.5, 0.5),
(0.5, 0.5, 0.5)),  # Normalize the images, each R,G,B value is normalized with
mean=0.5 and std=0.5
```

```
                                        ])


# Load the dataset using torchvision.datasets.ImageFolder and apply
transformations
dataset = datasets.ImageFolder(data_dir, transform=transform)

# Split the dataset into training and validation sets
train_size = int(0.9 * len(dataset))
valid_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, valid_size])

# Example of DataLoader creation for training and validation
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

print("Number of instance in train_set: %s" % len(train_dataset))
print("Number of instance in val_set: %s" % len(val_dataset))
```

```
Number of instance in train_set: 8519
Number of instance in val_set: 947
```

In [3]:
```
class_names = ['bird', 'bottle', 'bread', 'butterfly', 'cake', 'cat', 'chicken',
'cow', 'dog', 'duck',
                'elephant', 'fish', 'handgun', 'horse', 'lion', 'lipstick',
'seal', 'snake', 'spider', 'vase']
```

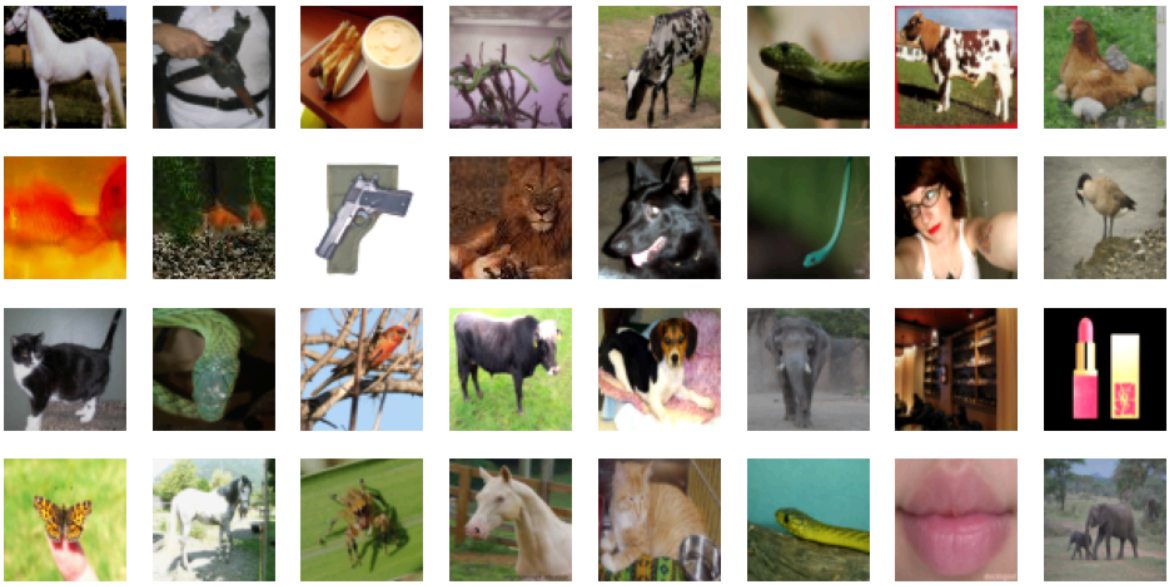
In [4]:
```
# obtain one batch of training images
dataiter = iter(train_loader)
images, labels = next(dataiter)
images = images.numpy() # convert images to numpy for display
```

In [5]:
```
import math

def imshow(img):
    img = img / 2 + 0.5  # unnormalize
    plt.imshow(np.transpose(img, (1, 2, 0)))  # convert from Tensor image

def visualize_data(images, categories, images_per_row = 8):
    class_names = ['bird', 'bottle', 'bread', 'butterfly', 'cake', 'cat',
'chicken', 'cow', 'dog', 'duck',
                'elephant', 'fish', 'handgun', 'horse', 'lion', 'lipstick',
'seal', 'snake', 'spider', 'vase']
    n_images = len(images)
    n_rows = math.ceil(float(n_images)/images_per_row)
    fig = plt.figure(figsize=(1.5*images_per_row, 1.5*n_rows))
    fig.patch.set_facecolor('white')
    for i in range(n_images):
        plt.subplot(n_rows, images_per_row, i+1)
        plt.xticks([])
        plt.yticks([])
        imshow(images[i])
        class_index = categories[i]
        plt.xlabel(class_names[class_index])
    plt.show()
```

In [6]:
```
visualize_data(images, labels)
```



For questions 3.1 to 3.5, you'll need to write your own model in a way that makes it easy for you to experiment with different architectures and parameters. The goal is to be able to pass the parameters to initialize a new instance of `YourModel` to build different network architectures with different parameters. Below are descriptions of some parameters for `YourModel`:

1. `Block confirguration`: Our network consists of many blocks. Each block has the pattern `[conv, batch norm, activation, conv, batch norm, activation, max pool, dropout]`. All convolutional layers have filter size $(3, 3)$, strides $(1, 1)$ and padding = 1, and all max pool layers have strides $(2, 2)$, kernel size 2, and padding = 0. The network will consists of a few blocks before applying a linear layer to output the logits for the softmax layer.

2. `list_feature_maps`: the number of feature maps in the blocks of the network. For example, if `list_feature_maps = [16, 32, 64]`, our network has two blocks with the input_channels or number of feature maps are `16, 32`, and `64` respectively.

3. `drop_rate`: the keep probability for dropout. Setting `drop_rate` to 0.0 means not using dropout.

4. `batch_norm`: the batch normalization function is used or not. Setting `batch_norm` to `false` means not using batch normalization.

5. `use_skip`: the skip connection is used in the blocks or not. Setting this to `true` means that we use `1x1` Conv2D with `strides=2` for the skip connection.

6. At the end, you need to apply `global average pooling (GAP)` (`AdaptiveAvgPool2d((1, 1))`) to flatten the 3D output tensor before defining the output linear layer for predicting the labels.

Here is the model confirguration of `YourCNN` if the `list_feature_maps = [16, 32, 64]` and `batch_norm = true`.

```
YourCNN(
  (block): ModuleList(
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=same)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): YourBlock(
      (block): ModuleList(
        (0): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1))
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1))
        (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): ReLU(inplace=True)
        (6): MaxPool2d(kernel_size=2, stride=2, padding=1, dilation=1, ceil_mode=False)
        (7): Dropout(p=0.2, inplace=False)
      )
      (skip_conv): Conv2d(16, 32, kernel_size=(1, 1), stride=(2, 2))
    )
    (4): YourBlock(
      (block): ModuleList(
        (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): ReLU(inplace=True)
        (6): MaxPool2d(kernel_size=2, stride=2, padding=1, dilation=1, ceil_mode=False)
        (7): Dropout(p=0.2, inplace=False)
      )
      (skip_conv): Conv2d(32, 64, kernel_size=(1, 1), stride=(2, 2))
    )
    (5): AdaptiveAvgPool2d(output_size=(1, 1))
    (6): Flatten(start_dim=1, end_dim=-1)
    (7): Linear(in_features=64, out_features=20, bias=True)
  )
)
```

Question 3.1: You need to implement the aforementioned CNN.

First, you need to implement the block of our CNN in the class `YourBlock`. You can ignore `use_skip` and `skip connection` for simplicity. However, you cannot earn full marks for this question.

[6 points]

In [7]:
```python
#Your code here
class YourBlock(nn.Module):
  def __init__(self, in_feature_maps, out_feature_maps, drop_rate = 0.2,
batch_norm = True, use_skip = True):
    super(YourBlock, self).__init__()
    self.use_skip = use_skip
    #Your code here
    # Block pattern: [conv, batch norm, activation, conv, batch norm,
activation, max pool, dropout]
    # conv: 3x3 filters, stride 1, padding 1 (keeps the spatial size); max pool:
kernel 2, stride 2, padding 0 (halves it)
    # If batch_norm is False the BatchNorm2d layers are replaced by Identity
(no-op) layers.
    bn = (lambda c: nn.BatchNorm2d(c)) if batch_norm else (lambda c:
nn.Identity())
```

```
    self.block = nn.ModuleList([
        nn.Conv2d(in_feature_maps, out_feature_maps, kernel_size=(3, 3), stride=
(1, 1), padding=1),
        bn(out_feature_maps),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_feature_maps, out_feature_maps, kernel_size=(3, 3),
stride=(1, 1), padding=1),
        bn(out_feature_maps),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2, padding=0),
        nn.Dropout(p=drop_rate),
    ])
    # Skip connection: a 1x1 convolution with stride 2 maps the block input [in,
H, W] to the
    # shape of the block output [out, H/2, W/2] so that the two can be added.
    if self.use_skip:
        self.skip_conv = nn.Conv2d(in_feature_maps, out_feature_maps,
kernel_size=(1, 1), stride=(2, 2))

  def forward(self, x):
    #Write your code here
    out = x
    for layer in self.block:
        out = layer(out)
    if self.use_skip:
        out = out + self.skip_conv(x)      # residual connection: block(x) +
skip(x)
    return out
```

Second, you need to use the above `YourBlock` to implement the class `YourCNN`.

[6 points]

In [9]:

```
class YourCNN(nn.Module):
  def __init__(self, list_feature_maps = [16, 32, 64], drop_rate = 0.2,
batch_norm= True, use_skip = True, num_classes = 20):
    super(YourCNN, self).__init__()
    layers = []

    # Stem: one 3x3 conv (3 -> list_feature_maps[0]) with batch norm and ReLU,
as in the model configuration above
    layers += [nn.Conv2d(3, list_feature_maps[0], kernel_size=(3, 3), stride=
(1, 1), padding='same'),
               nn.BatchNorm2d(list_feature_maps[0]) if batch_norm else
nn.Identity(),
               nn.ReLU(inplace=True)]
    # One YourBlock per pair of consecutive feature-map sizes: [16, 32, 64] ->
blocks 16 -> 32 and 32 -> 64
    for in_maps, out_maps in zip(list_feature_maps[:-1], list_feature_maps[1:]):
        layers.append(YourBlock(in_maps, out_maps, drop_rate=drop_rate,
batch_norm=batch_norm, use_skip=use_skip))
    # Global average pooling flattens the [C, H, W] tensor to a C-dimensional
vector,
    # followed by the output linear layer that produces the logits
    layers += [nn.AdaptiveAvgPool2d(output_size=(1, 1)),
               nn.Flatten(start_dim=1, end_dim=-1),
               nn.Linear(in_features=list_feature_maps[-1],
out_features=num_classes, bias=True)]
    self.block = nn.ModuleList(layers)
```

```
    self.num_features = list_feature_maps[-1]     # size of the penultimate
layer (used in Question 3.5)

  def features(self, x):
    """Output of the penultimate layer (the flattened global-average-pooled
feature vector)."""
    for layer in self.block[:-1]:
        x = layer(x)
    return x

  def forward(self, x):
    #Write your code here
    return self.block[-1](self.features(x))     # logits
```

We declare `my_cnn` from `YourCNN` as follows.

In [10]:
```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
my_cnn = YourCNN(list_feature_maps = [16, 32, 64], use_skip = True)
my_cnn = my_cnn.to(device)
print(my_cnn)
```

```
YourCNN(
  (block): ModuleList(
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=same)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): YourBlock(
      (block): ModuleList(
        (0): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (5): ReLU(inplace=True)
        (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (7): Dropout(p=0.2, inplace=False)
      )
      (skip_conv): Conv2d(16, 32, kernel_size=(1, 1), stride=(2, 2))
    )
    (4): YourBlock(
      (block): ModuleList(
        (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
        (5): ReLU(inplace=True)
        (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (7): Dropout(p=0.2, inplace=False)
      )
      (skip_conv): Conv2d(32, 64, kernel_size=(1, 1), stride=(2, 2))
    )
    (5): AdaptiveAvgPool2d(output_size=(1, 1))
    (6): Flatten(start_dim=1, end_dim=-1)
    (7): Linear(in_features=64, out_features=20, bias=True)
  )
)
```

We declare the optimizer and the loss function.

In [11]:
```
# Loss and optimizer
learning_rate = 0.001
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(my_cnn.parameters(), lr=learning_rate)
```

Here are the codes to compute the loss and accuracy.

In [12]:
```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

def compute_loss(model, loss_fn, loader):
  loss = 0
  # Set model to eval mode for inference
  model.eval()
```

```
  with torch.no_grad():  # No need to track gradients for validation
    for (batchX, batchY) in loader:
      # Move data to the same device as the model
      batchX, batchY = batchX.to(device).type(torch.float32),
batchY.to(device).type(torch.long)
      loss += loss_fn(model(batchX), batchY)
  # Set model back to train mode
  model.train()
  return float(loss)/len(loader)
```

In [13]:
```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

def compute_acc(model, loader):
    correct = 0
    totals = 0
    # Set model to eval mode for inference
    model.eval()
    for (batchX, batchY) in loader:
        # Move batchX and batchY to the same device as the model
        batchX, batchY = batchX.to(device).type(torch.float32),
batchY.to(device)
        outputs = model(batchX)  # feed batch to the model
        totals += batchY.size(0)  # accumulate totals with the current batch
size
        predicted = torch.argmax(outputs.data, 1)  # get the predicted class
        # Move batchY to the same device as predicted for comparison
        correct += (predicted == batchY).sum().item()
    return correct / totals
```

Here is the code to train our model.

In [14]:
```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

def fit(model= None, train_loader = None, valid_loader= None, optimizer = None,
        num_epochs = 50, verbose = True, seed= 1234):
  torch.manual_seed(seed)
  # Move the model to the device before initializing the optimizer
  model.to(device) # Move the model to the GPU

  if optimizer == None:
    optim = torch.optim.Adam(model.parameters(), lr = 0.001) # Now initialize
optimizer with model on GPU
  else:
    optim = optimizer
  history = dict()
  history['val_loss'] = list()
  history['val_acc'] = list()
  history['train_loss'] = list()
  history['train_acc'] = list()

  for epoch in range(num_epochs):
    model.train()
    for (X, y) in train_loader:
      # Move input data to the same device as the model
      X,y = X.to(device), y.to(device)
      # Forward pass
      outputs = model(X.type(torch.float32)) # X is already on the correct
device
      loss = loss_fn(outputs, y.type(torch.long))
```

```
      # Backward and optimize
      optim.zero_grad()
      loss.backward()
      optim.step()
      #losses and accuracies for epoch
    val_loss = compute_loss(model, loss_fn, valid_loader)
    val_acc = compute_acc(model, valid_loader)
    train_loss = compute_loss(model, loss_fn, train_loader)
    train_acc = compute_acc(model, train_loader)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    if not verbose: #verbose = True means we do show the training information
during training
      print(f"Epoch {epoch+1}/{num_epochs}")
      print(f"train loss= {train_loss:.4f} - train acc= {train_acc*100:.2f}% -
valid loss= {val_loss:.4f} - valid acc= {val_acc*100:.2f}%")
  return history
```

In [13]:

```
history = fit(model= my_cnn, train_loader=train_loader, valid_loader =
val_loader, optimizer = optimizer, num_epochs= 20, verbose = False)
```

```
Epoch 1/20
train loss= 2.5009 - train acc= 24.97% - valid loss= 2.4890 - valid acc= 23.44%
Epoch 2/20
train loss= 2.0733 - train acc= 33.78% - valid loss= 2.1433 - valid acc= 31.57%
Epoch 3/20
train loss= 1.9553 - train acc= 38.23% - valid loss= 2.0252 - valid acc= 36.85%
Epoch 4/20
train loss= 1.7638 - train acc= 45.35% - valid loss= 1.7995 - valid acc= 43.82%
Epoch 5/20
train loss= 1.8678 - train acc= 42.90% - valid loss= 1.8917 - valid acc= 41.82%
Epoch 6/20
train loss= 1.6110 - train acc= 48.84% - valid loss= 1.6684 - valid acc= 46.57%
Epoch 7/20
train loss= 1.5983 - train acc= 49.56% - valid loss= 1.6505 - valid acc= 49.52%
Epoch 8/20
train loss= 1.4415 - train acc= 55.07% - valid loss= 1.4806 - valid acc= 53.75%
Epoch 9/20
train loss= 1.4547 - train acc= 54.17% - valid loss= 1.5223 - valid acc= 52.69%
Epoch 10/20
train loss= 1.4915 - train acc= 52.67% - valid loss= 1.5969 - valid acc= 49.63%
Epoch 11/20
train loss= 1.4469 - train acc= 54.65% - valid loss= 1.5402 - valid acc= 50.90%
Epoch 12/20
train loss= 1.2956 - train acc= 58.95% - valid loss= 1.3658 - valid acc= 57.13%
Epoch 13/20
train loss= 1.3620 - train acc= 55.98% - valid loss= 1.4856 - valid acc= 53.22%
Epoch 14/20
train loss= 1.4002 - train acc= 54.28% - valid loss= 1.5115 - valid acc= 50.79%
Epoch 15/20
train loss= 1.2055 - train acc= 61.90% - valid loss= 1.2736 - valid acc= 59.98%
Epoch 16/20
train loss= 1.3081 - train acc= 58.15% - valid loss= 1.3760 - valid acc= 56.07%
Epoch 17/20
train loss= 1.2294 - train acc= 61.89% - valid loss= 1.3230 - valid acc= 59.56%
Epoch 18/20
train loss= 1.2808 - train acc= 60.89% - valid loss= 1.3754 - valid acc= 60.08%
Epoch 19/20
train loss= 1.2248 - train acc= 60.08% - valid loss= 1.3862 - valid acc= 56.18%
Epoch 20/20
train loss= 1.1880 - train acc= 61.22% - valid loss= 1.2803 - valid acc= 59.24%
```

Please note that you struggle in implementing the aforementioned CNN. You can use the MiniVGG network in our labs for doing the following questions. However, you cannot earn any mark for 3.1 and 3.2.

Question 3.2: Exploring Data Mixup Technique for Improving Generalization Ability.

[4 points]

Data mixup is another super-simple technique used to boost the generalization ability of deep learning models. You need to incoroporate data mixup technique to the above deep learning model and experiment its performance. There are some papers and documents for data mixup as follows:

- Main paper for data mixup link for main paper and a good article article link.

You need to extend your model developed above, train a model using data mixup, and write your observations and comments about the result.

In [15]:
```
#Your code here


def mixup_data(X, y, lam=0.75):
    #75% original image and 25 % mix image
    index = torch.randperm(X.size(0), device=X.device)
    X_mixed = lam * X + (1 - lam) * X[index]
    return X_mixed, y, y[index], lam

def mixup_batch_loss(model, X, y, lam= 0.75):
    X_mixed, y_a, y_b, lam = mixup_data(X, y, lam)
    logits = model(X_mixed)
    return lam * loss_fn(logits, y_a) + (1 - lam) * loss_fn(logits, y_b)


def fit_custom(model, train_loader, valid_loader, batch_loss_fn, num_epochs=10,
lr=0.001, seed=1234, verbose=True):
  """
  Same training loop as `fit` above (Adam, lr = 0.001), but the loss of each
mini-batch is computed by
  `batch_loss_fn(model, X, y)`, which lets us plug in Mixup / CutMix / the OVA
loss / the rotation loss.
  Validation and training metrics are always computed with the plain cross-
entropy and accuracy.
  """
  torch.manual_seed(seed)
  np.random.seed(seed)
  model.to(device)
  optim = torch.optim.Adam(model.parameters(), lr=lr)
  history = {'val_loss': [], 'val_acc': [], 'train_loss': [], 'train_acc': []}
  for epoch in range(num_epochs):
    model.train()
    for (X, y) in train_loader:
      X, y = X.to(device).type(torch.float32), y.to(device).type(torch.long)
      loss = batch_loss_fn(model, X, y)
      optim.zero_grad()
      loss.backward()
      optim.step()
    val_loss = compute_loss(model, loss_fn, valid_loader)
    val_acc = compute_acc(model, valid_loader)
    train_loss = compute_loss(model, loss_fn, train_loader)
    train_acc = compute_acc(model, train_loader)
    for k, v in zip(('val_loss', 'val_acc', 'train_loss', 'train_acc'),
(val_loss, val_acc, train_loss, train_acc)):
      history[k].append(v)
    if verbose:
      print(f"Epoch {epoch+1}/{num_epochs}")
      print(f"train loss= {train_loss:.4f} - train acc= {train_acc*100:.2f}% -
valid loss= {val_loss:.4f} - valid acc= {val_acc*100:.2f}%")
  return history

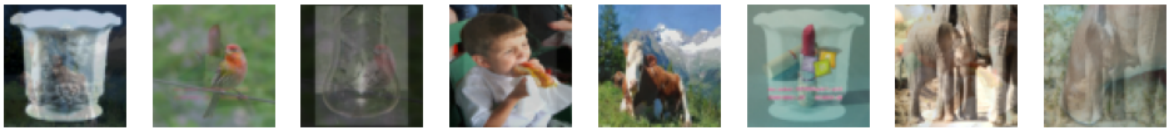

def summarize(name, h):
  return f"{name:28s} final valid acc= {h['val_acc'][-1]*100:.2f}%  best valid
acc= {max(h['val_acc'])*100:.2f}%  final valid loss= {h['val_loss'][-1]:.4f}
final train acc= {h['train_acc'][-1]*100:.2f}%"
```

```
# Visualise a few mixed images
X_vis, y_vis = next(iter(train_loader))
X_mix, _, _, lam_vis = mixup_data(X_vis[:8], y_vis[:8], lam=0.75)
print(f"Example of mixed images with lam = {lam_vis:.2f}")
visualize_data(X_mix.numpy(), y_vis[:8].numpy())

num_epochs = 20
my_cnn_mixup = YourCNN(list_feature_maps=[16, 32, 64], use_skip=True)
history_mixup = fit_custom(my_cnn_mixup, train_loader, val_loader,
mixup_batch_loss, num_epochs=num_epochs)

print()
print(summarize("Baseline (Q3.1)", history))
print(summarize("Mixup (lamda=0.75)", history_mixup))
```

```
Example of mixed images with lam = 0.75
```



```
Epoch 1/20
train loss= 2.3723 - train acc= 29.43% - valid loss= 2.4031 - valid acc= 26.82%
Epoch 2/20
train loss= 2.1715 - train acc= 32.13% - valid loss= 2.2005 - valid acc= 29.99%
Epoch 3/20
train loss= 2.0824 - train acc= 35.30% - valid loss= 2.1187 - valid acc= 35.37%
Epoch 4/20
train loss= 1.9504 - train acc= 41.03% - valid loss= 1.9662 - valid acc= 40.13%
Epoch 5/20
train loss= 1.8125 - train acc= 43.80% - valid loss= 1.8558 - valid acc= 42.45%
Epoch 6/20
train loss= 1.8097 - train acc= 44.86% - valid loss= 1.8396 - valid acc= 43.40%
Epoch 7/20
train loss= 1.7758 - train acc= 44.68% - valid loss= 1.8039 - valid acc= 44.46%
Epoch 8/20
train loss= 1.6550 - train acc= 48.93% - valid loss= 1.6565 - valid acc= 48.15%
Epoch 9/20
train loss= 1.5953 - train acc= 50.48% - valid loss= 1.6548 - valid acc= 49.10%
Epoch 10/20
train loss= 1.6954 - train acc= 48.05% - valid loss= 1.7311 - valid acc= 47.41%
Epoch 11/20
train loss= 1.5163 - train acc= 56.02% - valid loss= 1.5737 - valid acc= 54.49%
Epoch 12/20
train loss= 1.4815 - train acc= 56.77% - valid loss= 1.5166 - valid acc= 55.02%
Epoch 13/20
train loss= 1.5156 - train acc= 53.41% - valid loss= 1.5481 - valid acc= 54.28%
Epoch 14/20
train loss= 1.4210 - train acc= 57.35% - valid loss= 1.4861 - valid acc= 55.97%
Epoch 15/20
train loss= 1.5737 - train acc= 51.26% - valid loss= 1.6036 - valid acc= 50.79%
Epoch 16/20
train loss= 1.3950 - train acc= 58.76% - valid loss= 1.4408 - valid acc= 56.71%
Epoch 17/20
train loss= 1.3495 - train acc= 61.45% - valid loss= 1.4313 - valid acc= 59.03%
Epoch 18/20
train loss= 1.2983 - train acc= 62.65% - valid loss= 1.3753 - valid acc= 60.93%
Epoch 19/20
train loss= 1.3312 - train acc= 60.70% - valid loss= 1.4100 - valid acc= 58.50%
Epoch 20/20
train loss= 1.3095 - train acc= 61.37% - valid loss= 1.3913 - valid acc= 60.61%
```

Observations (Question 3.2). From observation we can see that the mixup technique actually makes the model worst in terms of loss. We did see a slight 1% improvement over in valid accuracy. However, this could just be training noise. Accordance with the original paper, model was trained on 200+ epoch and if we compare the result at 20 epoch. The result of this training session on 20 epoch accurately replicated what occured in the original paper. In the original paper we obsrver that under 50 epoch mixup is expected to perform worst on average. It is very likely to be true in our case as well as we have not reached a minimum during our training.

Question 3.3: Exploring CutMix Technique for Improving Generalization Ability.

[4 points]

CutMix is another super-simple technique used to boost the generalization ability of deep learning models. You need to incoroporate data CutMix technique to the above deep

learning model and experiment its performance. There are some papers and documents for data mixup as follows:

- Main paper for Cutmix link for main paper and a good article article link.

You need to extend your model developed above, train a model using data CutMix, and write your observations and comments about the result.

In [16]:
```
#Your code here

def rand_bbox(H, W, lam):
    cut_ratio = np.sqrt(1.0 - lam)
    cut_h, cut_w = int(H * cut_ratio), int(W * cut_ratio)
    cy, cx = np.random.randint(H), np.random.randint(W)           # centre of
the box, uniform over the image
    y1, y2 = np.clip(cy - cut_h // 2, 0, H), np.clip(cy + cut_h // 2, 0, H)
    x1, x2 = np.clip(cx - cut_w // 2, 0, W), np.clip(cx + cut_w // 2, 0, W)
    return y1, y2, x1, x2

def cutmix_data(X, y, alpha=1.0):
    """Returns the cut-and-pasted inputs, the two label vectors and the adjusted
lam."""
    lam = np.random.beta(alpha, alpha) if alpha > 0 else 1.0
    index = torch.randperm(X.size(0), device=X.device)
    H, W = X.size(2), X.size(3)
    y1, y2, x1, x2 = rand_bbox(H, W, lam)
    X_cut = X.clone()
    X_cut[:, :, y1:y2, x1:x2] = X[index, :, y1:y2, x1:x2]        # paste the
patch of image B into image A
    lam = 1 - ((y2 - y1) * (x2 - x1) / (H * W))                   # adjust lam
to the exact area after clipping
    return X_cut, y, y[index], lam

def cutmix_batch_loss(model, X, y, alpha=1.0):
    X_cut, y_a, y_b, lam = cutmix_data(X, y, alpha)
    logits = model(X_cut)
    return lam * loss_fn(logits, y_a) + (1 - lam) * loss_fn(logits, y_b)


# Visualise a few CutMix images
X_cut_vis, _, _, lam_vis = cutmix_data(X_vis[:8], y_vis[:8], alpha=1.0)
print(f"Example of CutMix images with lam = {lam_vis:.2f}")
visualize_data(X_cut_vis.numpy(), y_vis[:8].numpy())

# Train the same CNN with CutMix (alpha = 1.0 as in the paper)
my_cnn_cutmix = YourCNN(list_feature_maps=[16, 32, 64], use_skip=True)
history_cutmix = fit_custom(my_cnn_cutmix, train_loader, val_loader,
cutmix_batch_loss, num_epochs=num_epochs)

print()
print(summarize("Baseline (Q3.1)", history))
print(summarize("Mixup (lamda=0.75)", history_mixup))
print(summarize("CutMix (alpha=1.0)", history_cutmix))
```

```
Example of CutMix images with lam = 0.63
```



```
Epoch 1/20
train loss= 2.3540 - train acc= 27.02% - valid loss= 2.3573 - valid acc= 26.72%
Epoch 2/20
train loss= 2.2095 - train acc= 30.73% - valid loss= 2.2488 - valid acc= 26.93%
Epoch 3/20
train loss= 2.0708 - train acc= 35.67% - valid loss= 2.1307 - valid acc= 33.47%
Epoch 4/20
train loss= 1.9292 - train acc= 40.91% - valid loss= 1.9678 - valid acc= 38.33%
Epoch 5/20
train loss= 1.9746 - train acc= 38.60% - valid loss= 1.9952 - valid acc= 36.75%
Epoch 6/20
train loss= 1.8728 - train acc= 41.59% - valid loss= 1.9015 - valid acc= 41.71%
Epoch 7/20
train loss= 1.8264 - train acc= 42.04% - valid loss= 1.9242 - valid acc= 38.86%
Epoch 8/20
train loss= 1.6639 - train acc= 49.64% - valid loss= 1.6891 - valid acc= 47.73%
Epoch 9/20
train loss= 1.6104 - train acc= 48.62% - valid loss= 1.6599 - valid acc= 46.88%
Epoch 10/20
train loss= 1.5764 - train acc= 51.00% - valid loss= 1.6200 - valid acc= 48.36%
Epoch 11/20
train loss= 1.6304 - train acc= 50.70% - valid loss= 1.7038 - valid acc= 47.73%
Epoch 12/20
train loss= 1.4669 - train acc= 54.04% - valid loss= 1.5208 - valid acc= 52.80%
Epoch 13/20
train loss= 1.5572 - train acc= 50.70% - valid loss= 1.6333 - valid acc= 46.99%
Epoch 14/20
train loss= 1.6357 - train acc= 49.07% - valid loss= 1.7171 - valid acc= 44.46%
Epoch 15/20
train loss= 1.4242 - train acc= 55.64% - valid loss= 1.4803 - valid acc= 55.12%
Epoch 16/20
train loss= 1.4479 - train acc= 54.43% - valid loss= 1.5029 - valid acc= 53.54%
Epoch 17/20
train loss= 1.4253 - train acc= 55.26% - valid loss= 1.4802 - valid acc= 54.38%
Epoch 18/20
train loss= 1.2822 - train acc= 60.97% - valid loss= 1.3626 - valid acc= 58.18%
Epoch 19/20
train loss= 1.4495 - train acc= 55.32% - valid loss= 1.5566 - valid acc= 54.07%
Epoch 20/20
train loss= 1.3114 - train acc= 58.74% - valid loss= 1.4028 - valid acc= 55.54%
```

As expected, the same issue persist for this training as well. In the original paper we see that top 1% error is only better at around 150 epoch for cutmix training compare to baseline model. It is expected that the model would need longer training time since we are making the dataset more complex thus it need longers time to optimise the parameters. Because we are also training on different and robust data we expect the model to generalize better. Because the lost on cutmix is based on how much % of x1 and x2, we expect the model to be better at identifying key features of the data. Within the original paper, this technique is an improvement to cutout. The improvement mainly comes from the fact that you are able to train the model on the cutout region as well, based on cam.

Question 3.4: Implement the one-versus-all (OVA) loss

The details are as follows:

- You need to apply `the sigmoid activation function` to logits $h = [h_1, h_2, \ldots, h_M]$ instead of `the softmax activation` function as usual to obtain $p = [p_1, p_2, \ldots, p_M]$, meaning that $p_i = sigmoid(h_i), i = 1, \ldots, M$. Note that M is the number of classes.
- Given a data example x with the ground-truth label y, the idea is to maximize the likelihood p_y and to minimize the likelihoods $p_i, i \neq y$. Therefore, the objective function is to find the model parameters to
  - $\max \left\{ \log p_y + \sum_{i \neq y} \log(1 - p_i) \right\}$ or equivalently $\min \left\{ -\log p_y - \sum_{i \neq y} \log(1 - p_i) \right\}$.
  - For example, if $M = 3$ and $y = 2$, you need to minimize $\min \{ -\log(1 - p_1) - \log p_2 - \log(1 - p_3) \}$.

Compare the model trained with the OVA loss and the same model trained with the standard cross-entropy loss.

[4 points]

In [18]:
```
#Your code here


def ova_loss(logits, y):
    """
    One-versus-all loss:  loss(h, y) = -log p_y - sum_{i != y} log(1 - p_i)
with   p_i = sigmoid(h_i)

    logits h has shape [batch_size, M]; each class i gets its own binary
classifier "class i vs the rest",
    so we push p_y towards 1 and every other p_i towards 0. Averaged over the
mini-batch.
    """
    log_p = torch.nn.functional.logsigmoid(logits)            # log p_i = log
sigmoid(h_i)          (numerically stable)
    log_1_minus_p = torch.nn.functional.logsigmoid(-logits)   # log(1 - p_i) =
log sigmoid(-h_i)
    one_hot = torch.nn.functional.one_hot(y, num_classes=logits.size(1)).float()
# 1 at position y, 0 elsewhere
```

```
    per_example = -(one_hot * log_p + (1 - one_hot) * log_1_minus_p).sum(dim=1)
# -log p_y - sum_{i!=y} log(1-p_i)
    return per_example.mean()

def ova_batch_loss(model, X, y):
    return ova_loss(model(X), y)

# Train the same CNN with the OVA loss; the prediction is still argmax_i p_i =
argmax_i h_i
my_cnn_ova = YourCNN(list_feature_maps=[16, 32, 64], use_skip=True)
history_ova = fit_custom(my_cnn_ova, train_loader, val_loader, ova_batch_loss,
num_epochs=10)


print(summarize("OVA loss", history_ova))
print(f"OVA loss of the OVA model on the validation set:
{compute_loss(my_cnn_ova, ova_loss, val_loader):.4f}")
```

```
Epoch 1/10
train loss= 2.3045 - train acc= 31.25% - valid loss= 2.3386 - valid acc= 28.41%
Epoch 2/10
train loss= 2.1844 - train acc= 31.20% - valid loss= 2.2197 - valid acc= 28.30%
Epoch 3/10
train loss= 1.9955 - train acc= 37.08% - valid loss= 2.0371 - valid acc= 34.95%
Epoch 4/10
train loss= 1.8324 - train acc= 43.43% - valid loss= 1.8679 - valid acc= 43.19%
Epoch 5/10
train loss= 1.7857 - train acc= 44.21% - valid loss= 1.8100 - valid acc= 42.56%
Epoch 6/10
train loss= 2.0358 - train acc= 38.10% - valid loss= 2.0415 - valid acc= 35.80%
Epoch 7/10
train loss= 1.6589 - train acc= 48.29% - valid loss= 1.7229 - valid acc= 45.09%
Epoch 8/10
train loss= 1.5199 - train acc= 52.05% - valid loss= 1.5357 - valid acc= 49.21%
Epoch 9/10
train loss= 1.4543 - train acc= 54.70% - valid loss= 1.5024 - valid acc= 53.43%
Epoch 10/10
train loss= 1.5907 - train acc= 49.48% - valid loss= 1.6492 - valid acc= 47.41%
OVA loss                    final valid acc= 47.41%  best valid acc= 53.43%
final valid loss= 1.6492  final train acc= 49.48%
OVA loss of the OVA model on the validation set: 2.5677
```

Question 3.5: Incorporate the unsupervised loss to the cross-entropy loss

The unsupervised loss is defined as follows

- For each mini-batch x of images, we rotate these images by $0^\circ, 90^\circ, 180^\circ, 270^\circ$ and then predict for labels $0, 1, 2, 3$ respectively. For example, if we rotate an image by 180°, we need to predict the ground-truth label 2.
- In terms of architecture, on the top of the penultimate layer (i.e., the layer right before the current output layer) of your CNN, you add one more output layer to predict the rotation angles (0° - label 0, 90° - label 1, 180° - label 2, and 270° - label 3).
- The final loss is $total_loss = CE_loss + \lambda \times unsupervised_loss$ where $\lambda > 0$ is a trade-off parameter.
- You can set λ (e.g., $\lambda = 0.1$) a value and train a model with this value.

- Note that you can as many cells as you want to extend the current code.

[6 points]

In [19]:
```
#Your code here


class RotationCNN(YourCNN):
    """YourCNN with an extra output layer that predicts the rotation angle (4
classes)."""
    def __init__(self, list_feature_maps=[16, 32, 64], drop_rate=0.2,
batch_norm=True, use_skip=True, num_classes=20):
        super().__init__(list_feature_maps, drop_rate, batch_norm, use_skip,
num_classes)
        self.rotation_head = nn.Linear(self.num_features, 4)

    def forward_rotation(self, x):
        return self.rotation_head(self.features(x))      # rotation logits from
the penultimate layer

def rotate_batch(X):
    """Returns the 4 rotated copies of X (0, 90, 180, 270 degrees) and their
rotation labels 0, 1, 2, 3."""
    X_rot = torch.cat([torch.rot90(X, k, dims=(2, 3)) for k in range(4)],
dim=0)
    r = torch.arange(4, device=X.device).repeat_interleave(X.size(0))
    return X_rot, r

lambda_rot = 0.1

def rotation_batch_loss(model, X, y, lam=lambda_rot):
    ce_loss = loss_fn(model(X), y)                       # supervised loss on
the original images
    X_rot, r = rotate_batch(X)
    unsupervised_loss = loss_fn(model.forward_rotation(X_rot), r)   # rotation
prediction on the rotated images
    return ce_loss + lam * unsupervised_loss

# Show the four rotations of the first two images of a batch (labels shown are
the class labels)
X_rot_vis, r_vis = rotate_batch(X_vis[:2])
print("Rotations 0, 90, 180, 270 degrees of two training images (rotation
labels:", r_vis.tolist(), ")")
visualize_data(X_rot_vis.numpy(), y_vis[:2].repeat(4).numpy(), images_per_row=8)

my_cnn_rot = RotationCNN(list_feature_maps=[16, 32, 64], use_skip=True)
history_rot = fit_custom(my_cnn_rot, train_loader, val_loader,
rotation_batch_loss, num_epochs=10)

# accuracy of the auxiliary rotation task on the validation set
@torch.no_grad()
def rotation_acc(model, loader):
    model.eval(); correct, total = 0, 0
    for X, _ in loader:
        X_rot, r = rotate_batch(X.to(device).type(torch.float32))
        correct += (model.forward_rotation(X_rot).argmax(1) == r).sum().item();
total += r.numel()
    return correct / total
```

```
print()
print(summarize("Baseline CE (Q3.1)", history))
print(summarize("Mixup", history_mixup))
print(summarize("CutMix", history_cutmix))
print(summarize("OVA loss", history_ova))
print(summarize(f"CE + {lambda_rot} * rotation loss", history_rot))
print(f"Rotation-prediction accuracy of the auxiliary head on the validation
set: {rotation_acc(my_cnn_rot, val_loader)*100:.2f}%")
```

```
Rotations 0, 90, 180, 270 degrees of two training images (rotation labels: [0, 0,
1, 1, 2, 2, 3, 3] )
```



```
Epoch 1/20
train loss= 2.2425 - train acc= 31.39% - valid loss= 2.2571 - valid acc= 29.36%
Epoch 2/20
train loss= 2.0642 - train acc= 33.67% - valid loss= 2.1087 - valid acc= 30.41%
Epoch 3/20
train loss= 1.8265 - train acc= 43.10% - valid loss= 1.8555 - valid acc= 42.34%
Epoch 4/20
train loss= 1.6944 - train acc= 46.48% - valid loss= 1.7268 - valid acc= 43.51%
Epoch 5/20
train loss= 1.7603 - train acc= 43.75% - valid loss= 1.7865 - valid acc= 43.40%
Epoch 6/20
train loss= 1.5832 - train acc= 49.72% - valid loss= 1.5979 - valid acc= 49.00%
Epoch 7/20
train loss= 1.4783 - train acc= 53.00% - valid loss= 1.5337 - valid acc= 52.90%
Epoch 8/20
train loss= 1.4120 - train acc= 55.19% - valid loss= 1.4257 - valid acc= 55.97%
Epoch 9/20
train loss= 1.4996 - train acc= 52.96% - valid loss= 1.5664 - valid acc= 49.84%
Epoch 10/20
train loss= 1.3828 - train acc= 55.48% - valid loss= 1.4570 - valid acc= 54.80%
Epoch 11/20
train loss= 1.3375 - train acc= 58.13% - valid loss= 1.4311 - valid acc= 55.12%
Epoch 12/20
train loss= 1.2567 - train acc= 60.32% - valid loss= 1.3249 - valid acc= 58.50%
Epoch 13/20
train loss= 1.4378 - train acc= 53.45% - valid loss= 1.5592 - valid acc= 51.85%
Epoch 14/20
train loss= 1.2562 - train acc= 59.70% - valid loss= 1.3533 - valid acc= 56.81%
Epoch 15/20
train loss= 1.1549 - train acc= 63.26% - valid loss= 1.2316 - valid acc= 61.99%
Epoch 16/20
train loss= 1.1777 - train acc= 62.47% - valid loss= 1.2387 - valid acc= 59.45%
Epoch 17/20
train loss= 1.0738 - train acc= 65.38% - valid loss= 1.1825 - valid acc= 63.25%
```

Question 3.6 (Kaggle competition)

[15 points]

In [ ]:
```
%%sql
```

You can reuse the best model obtained in this assignment or develop new models to evaluate on the **testing set of the assignment Kaggle competion**. However, to gain any

points for this question, your testing accuracy must **exceed** the accuracy threshold from a base models developed by us as shown in the leader board of the competition.

The marks for this question are as follows:

- If you *beat* the first boss model, you gain *6 points*.
- If you *beat* the second boss model, you gain *9 points*.
- If you *beat* the third boss model, you gain *12 points*.
- If you *beat* the fourth boss model, you gain *15 points*.

Moreover, if you rank in the top 10 of the entire cohort, you will receive a prestigious certificate signed by a renowned professor from our Faculty of Information Technology — a valuable highlight for your CV.

Link to the Kaggle Starter Notebook:
https://colab.research.google.com/drive/1U_baRDfSQZu2pMXYfMDjkYaaD7Kc3KVF?usp=sharing

Link to the Kaggle competition site:

- For FIT5215 Clayton cohort:
  https://www.kaggle.com/t/a451c4074bcb4d6da5735446dc8f6005
- For FIT5215 Malaysia cohort:
  https://www.kaggle.com/t/01432633e5a2459baab213bec6d98bc8

END OF ASSIGNMENT
GOOD LUCK WITH YOUR ASSIGNMENT 1!